

머신러닝 8 교시

단순선형회귀분석

- 회귀의 뜻에서 잔차분석까지 -

단순선형회귀는 통계학과 머신러닝의 만남점에서 가장 먼저 배우는 모델이다. 알고리즘이 단순하지만 그 안에 회귀계수 해석·가정 검토·잔차분석 같은 회귀의 모든 기초가 들어 있다. 본 강의를 마치면 다음을 할 수 있어야 한다.

학습 목표

- 설명변수·반응변수, 상관분석과 회귀분석의 차이를 구분할 수 있다.
- "회귀(regression)" 라는 이름의 어원을 골턴-피어슨 라인으로 설명할 수 있다.
- 결정계수 R^2 의 정의·해석· $SST=SSR+SSE$ 분해를 설명할 수 있다.
- 회귀계수의 통계적 유의성과 p-value 를 해석할 수 있다.
- 최소제곱법(OLS) 의 직관과 수식 유도를 설명할 수 있고 OLS 의 약점을 안다.
- 단순선형회귀 모형식 $y = \beta_0 + \beta_1 x + \epsilon$ 의 각 항이 무엇을 의미하는지 안다.
- scikit-learn 과 statsmodels 로 단순선형회귀를 학습·예측·요약할 수 있다.
- 가정이 깨졌을 때 데이터 변환(제곱근·로그) 으로 모형을 개선할 수 있다.
- 잔차분석으로 회귀의 4 가지 가정(선형성·정규성·등분산성·독립성) 을 검토할 수 있다.

목차

1 장. 회귀(Regression)란 무엇인가

- 1.1 회귀라는 말은 어디서 왔는가
- 1.2 회귀분석의 핵심 용어
- 1.3 회귀분석은 어디에 쓰이는가

2 장. 단순선형회귀의 개념

- 2.1 단순선형회귀의 모형식
- 2.2 회귀계수 β 의 해석
- 2.3 회귀분석의 기본 가정

3 장. 회귀계수를 찾는 법 — 최소제곱법(OLS)

- 3.1 "잘 설명한다"의 정의

- 3.2 세 직선의 비교 — OLS의 직관
- 3.3 OLS 수식 유도
- 3.4 OLS의 약점 — 이상치에 약하다

4 장. 회귀식 해석하기

- 4.1 cars 데이터 소개
- 4.2 회귀계수표(Coefficients) 읽는 법
- 4.3 통계적 유의성과 p-value

5 장. 모형의 적합도 평가 — 결정계수 R^2

- 5.1 직선이 데이터를 "설명한다"는 것
- 5.2 제곱합 분해: $SST = SSR + SSE$
- 5.3 결정계수 R^2 의 정의와 해석

6 장. Python으로 실습하는 단순선형회귀

- 6.1 데이터 살펴보기
- 6.2 상관분석 먼저
- 6.3 sklearn으로 모형 적합
- 6.4 model 객체 파헤치기
- 6.5 predict()로 예측하기

7 장. 잔차분석 — 가정 검토

- 7.1 왜 잔차를 봐야 하는가
- 7.2 잔차도 (Residuals vs Fitted)
- 7.3 정규성 검토 — Q-Q plot
- 7.4 등분산성 검토
- 7.5 가정이 깨졌을 때

8 장. 회귀분석 다시 하기 — 데이터 변환

- 8.1 왜 제곱근 변환인가
- 8.2 변환 후 다시 적합
- 8.3 물리적 의미 — 속도의 제곱
- 8.4 다른 변환 방식들

9 장. 정리

```
# cars 데이터 - speed(mph) vs dist(ft), 50 개 관측치
import pandas as pd

data = pd.DataFrame({
    'speed': [4,4,7,7,8,9,10,10,10,11,11,12,12,12,12,13,13,13,13,
              14,14,14,14,15,15,15,16,16,17,17,17,18,18,18,18,19,19,19,
              20,20,20,20,20,22,23,24,24,24,24,25],
    'dist': [2,10,4,22,16,10,18,26,34,17,28,14,20,24,28,26,34,34,46,
             26,36,60,80,20,26,54,32,40,32,40,50,42,56,76,84,36,46,68,
             32,48,52,56,64,66,54,70,92,93,120,85]
})

print(data.describe())
print(f"speed-dist 상관계수: {data['speed'].corr(data['dist']):.4f}")
```

=== scikit-learn 방식 ===

```
from sklearn.linear_model import LinearRegression
X = data[['speed']].values
y = data['dist'].values

model = LinearRegression()
model.fit(X, y)
print(f'절편  $\beta_0$  : {model.intercept_:.4f}')
print(f'기울기  $\beta_1$  : {model.coef_[0]:.4f}')
print(f'R2 : {model.score(X, y):.4f}')

# === sklearn 으로 예측 ===
import numpy as np
new_X = np.array([[21]]) # speed = 21 mph 일 때의 제동거리 예측
pred_sk = model.predict(new_X)
print(f'speed=21 mph 의 dist 예측 (sklearn): {pred_sk[0]:.4f} ft')
```

=== statsmodels 방식 ===

```
import statsmodels.api as sm
X_const = sm.add_constant(X)
model_sm = sm.OLS(y, X_const).fit()
print(model_sm.summary())

# === statsmodels 로 예측 + 95% 신뢰/예측구간 ===
new_X_const = sm.add_constant(new_X, has_constant='add')
pred = model_sm.get_prediction(new_X_const)
mean = pred.predicted_mean[0]
ci_low, ci_high = pred.conf_int()[0] # 평균 신뢰구간
obs_low, obs_high = pred.conf_int(obs=True)[0] # 개별 관측치 예측구간
print(f'평균 예측값 : {mean:.4f}')
print(f'95% 평균 신뢰구간 : [{ci_low:.4f}, {ci_high:.4f}]')
print(f'95% 예측구간(관측치) : [{obs_low:.4f}, {obs_high:.4f}]')
```

1 장. 회귀(Regression)란 무엇인가

회귀분석은 통계학 교과서의 첫 모델로 등장한다. 그러나 이 모델이 왜 '회귀'라는 독특한 이름을 갖게 되었는지를 알고 있는 사람은 의외로 적다. 본 장은 회귀의 어원과 역사부터 시작해, 변수의 이름이 왜 그렇게 많은지, 그리고 상관분석과 회귀분석은 무엇이 다른지까지 차근차근 풀어 가는 것을 목표로 한다.

1.1 회귀라는 말은 어디서 왔는가

1.1.1 골턴과 피어슨, 그리고 "평균으로의 회귀"

회귀(regression)라는 단어를 처음 통계에 가져온 사람은 프랜시스 골턴(Francis Galton, 1822~1911)이다. 그는 부모의 키와 자녀의 키를 함께 측정해 보고, '키가 매우 큰 부모의 자녀는 부모보다 약간 작아지고, 키가 매우 작은 부모의 자녀는 부모보다 약간 커지는' 경향을 발견했다. 자녀의 키가 인류 평균 쪽으로 '회귀'하는 것처럼 보였기 때문에, 그는 이 현상에 'regression toward mediocrity', 즉 '평균으로의 회귀'라는 이름을 붙였다. 이 아이디어를 수학적으로 다듬어 회귀분석을 학문적 도구로 만든 사람은 그의 제자 칼 피어슨(Karl Pearson, 1857~1936)이다. 우리가 지금 쓰는 상관계수(correlation coefficient)와 결정계수(coefficient of determination)는 모두 골턴-피어슨 라인의 유산이다. 현대에 있어, '회귀'라는 이름은 더 이상 '평균으로의 복귀'를 의미하지 않고, 한 변수에서 다른 변수로 이어지는 선형 관계를 추정하는 일반적인 분석 기법을 가리키게 되었다. 그래서 회귀선을 '추세선'이라고 한다.

'회귀'라는 단어에 발이 묶이지 말 것

현대의 회귀분석은 더 이상 '평균으로의 회귀'를 다루는 분석이 아니다. 두 변수 사이의 선형 관계를 추정하는 도구일 뿐이며, 'regression'이라는 단어는 역사적 흔적에 가깝다.

1.2 회귀분석의 핵심 용어

회귀분석을 공부하다 보면 같은 변수에 대해 너무 많은 이름이 나와 혼란스러울 수 있다. 독립변수, 설명변수, 예측변수, 입력변수, 특징 — 이 다섯이 모두 같은 개념이다. 이름이 많아진 이유는 분야마다 회귀분석을 가져다 쓰면서 자기들 식 이름을 붙였기 때문이다. 통계학에서는 '설명변수', 머신러닝에서는 '특징(feature)' 또는 '입력', 실험과학에서는 '독립변수'라고 부르는 식이다.

1.2.1 설명변수 vs 반응변수

[정의] 회귀분석의 두 종류 변수

• 설명변수(Explanatory variable) = 독립변수 = 예측변수 = 입력변수 = 특징(feature)

원인으로 간주되는 변수. 종속변수에 선행하며, 종속변수의 변화를 예측할 수 있다고 여겨지는 변수.

• 반응변수(Response variable) = 종속변수(Dependent variable)

설명변수의 변화에 의해 영향을 받을 것으로 기대되는 변수. 결과로 간주되는 변수.

1.2.2 변수 결정의 기준 – 이론·선행연구·직관

두 변수 중 어느 쪽을 설명변수로 두고 어느 쪽을 반응변수로 들지는 통계가 결정해 주지 않는다. 이는 분석가가 선행연구·이론적 배경·도메인 직관을 바탕으로 결정해야 하는 사안이다. 변수를 거꾸로 두면 같은 데이터로 회귀분석을 해도 결과 해석이 완전히 달라진다.

1.3 회귀분석은 어디에 쓰이는가

1977 년 미국 50 개 주의 살인사건 수를 종속변수로 두고, 인구·소득·문맹률·기대수명·고졸률·결빙일·면적과 같은 후보 설명변수 중에서 무엇이 살인사건 수와 가장 강한 선형 관계를 갖는지 탐색하는 것이 회귀분석의 전형적인 사용 예다. 모든 후보를 한꺼번에 넣으면 다중선형회귀가 되고, 그중 하나만 골라 단순한 직선 관계를 본다면 그것이 단순선형회귀가 된다.

1.3.1 미국 50 개 주 사례



변수 선택법

☑ 관심 있는 데이터에 대한 선행연구

☑ 이론적인 배경

☑ 분석가의 직관



위 사항들을 종합적으로 고려하여 종속 변수 y 와 설명 변수 x 를 결정
이게 뒤바뀌면 ...

[그림 2] '회귀분석의 목표'

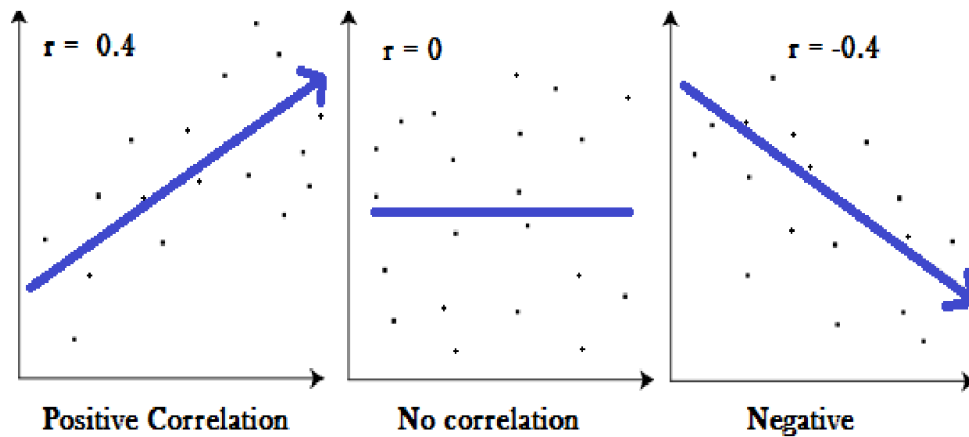
1.3.2 상관분석과 회귀분석의 결정적 차이

상관분석과 회귀분석은 비슷해 보이지만 결정적인 차이가 있다. 상관분석은 두 변수 사이의 '같이 움직이는 정도'만 측정한다. 즉 한 변수가 증가할 때 다른 변수가 함께 증가/감소하는 경향만 본다. 인과관계는 묻지 않는다. 반면 회귀분석은 '한 변수가 다른 변수를 설명한다'는 방향성을 전제로 한다. 비록 통계만으로 인과를 증명할 수는 없지만, 분석의 출발점에서 ' $x \rightarrow y$ ' 방향을 가정하는 것이다.

표본상관계수의 성질

- (1) $-1 \leq r \leq 1$
- (2) $|r|$ 의 값이 1에 가까울수록 강한 상관관계
 $|r|$ 의 값이 0에 가까울수록 약한 상관관계

여러 가지 경우의 산점도와 표본 상관계수



source : <http://www.statisticshowto.com/wp-content/uploads/2012/10/pearson-2-small.png> 서울대학교 통계학실험강의자료 | Rights Reserved.

1 장 정리

회귀라는 이름은 골턴의 '평균으로의 회귀'에서 비롯되었으나, 지금은 선형 관계 추정의 일반 도구를 의미한다.

설명변수(x)와 반응변수(y)의 구분은 통계가 아니라 도메인 지식이 결정한다.

상관분석은 '같이 움직임'만 보고, 회귀분석은 ' $x \rightarrow y$ ' 방향과 함수 관계를 추정한다.

2 장. 단순선형회귀의 개념

단순선형회귀(Simple Linear Regression)는 가장 단순한 형태의 회귀분석이다. 설명변수가 단 하나, 그리고 그 관계가 직선이라고 가정한다. 이 장에서는 단순선형회귀의 모형식이 무엇을 의미하는지, 회귀계수 β 를 어떻게 해석해야 하는지, 그리고 회귀분석이 깔고 있는 여러 가정들이 왜 중요한지를 다룬다. 가정 부분은 7 장의 잔차분석과 직접 이어진다.

2.1 단순선형회귀의 모형식

[모형식] 모집단 모형 vs 표본 회귀식

모집단 회귀모형: $y = \beta_0 + \beta_1 \cdot x + \epsilon$

여기서 ϵ 은 오차항(error term)이며, 우리가 직접 관측할 수 없는 추상적 개념이다.

표본 회귀식: $\hat{y} = \beta_0 + \beta_1 \cdot x$

실제 데이터로부터 추정한 직선이다. 햇(hat) 표시는 '추정값'을 뜻한다.

2.1.1 모집단 모형 vs 표본 회귀식 (ϵ 의 의미)

모집단 모형에는 ϵ 이라는 오차항이 등장한다. 이 오차항은 '직선으로는 설명할 수 없는 모든 것'을 모아 놓은 통계적 쓰레기통이다. 측정 오차, 누락된 변수의 영향, 본질적 변동성 — 이 모두가 ϵ 에 들어간다. 그런데 ϵ 은 결코 관측할 수 없다. 우리가 손에 쥔 데이터로부터 직접 만질 수 있는 것은 '예측값과 실제값의 차이', 즉 잔차(residual) 뿐이다.

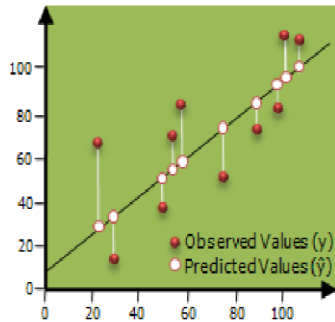
이 미묘한 차이를 헷갈리지 말자. 오차(error) ϵ 은 이론적 존재이고, 잔차(residual) $e = y - \hat{y}$ 는 데이터로부터 계산되는 실체다. 회귀분석은 잔차를 통해 오차의 성질을 간접적으로 추측하는 작업이라고도 할 수 있다.

2.1.2 단순선형회귀 vs 다중선형회귀 — 직선과 평면

설명변수가 하나면 단순선형회귀, 둘 이상이면 다중선형회귀가 된다. 설명변수가 하나일 때는 점들을 가장 잘 설명하는 '직선'을 찾고, 설명변수가 둘일 때는 '평면'을, 셋 이상이면 고차원의 '초평면(hyperplane)'을 찾는다. 추정의 본질은 같다 — 데이터와 모형 사이의 거리(제곱합)를 최소화하는 계수를 찾는 것이다.

회귀 계수의 추정

- 회귀분석의 목적: 종속변수 Y와 설명변수 X 사이의 관계를 선형으로 가정하고 이를 가장 잘 설명하는 회귀 계수(coefficients)를 추정하는 것
 - > 여기서 '잘 설명한다'는 의미는? 어떤 기준에서 '잘 설명'하는 것인가?
 - > '적합된 직선'과 '실제 데이터' 사이에 **y축 거리**(의 제곱)의 합을 최소화
 - > 최소제곱법(OLS; Ordinary Least Squares)



© Copyrights 2018. WeDataLab All Rights Reserved.

[그림 5] 상관계수와 회귀계수의 관계.

2.2 회귀계수 β 의 해석

2.2.1 기울기 β_1 : "x가 1 증가할 때 y는 얼마나"

회귀계수 β_1 은 단순선형회귀의 주인공이다. 이 값은 '설명변수 x가 한 단위(1) 증가할 때 반응변수 y가 평균적으로 얼마나 변하는지'를 알려 주는 지표다. 예를 들어 $\beta_1 = 3.93$ 이면 x가 1 늘어날 때 y의 평균이 3.93 증가한다는 뜻이다. 단위가 매우 중요하다 — speed가 mph 단위이고 dist가 ft 단위이면 β_1 의 단위는 'ft 당 mph'다.

회귀 계수의 추정

- 최소제곱법(OLS; Ordinary Least Squares)

$$\text{실제 } y \text{ 값: } y = \beta_0 + \beta_1 x + \varepsilon$$

$$\text{예측된 } y \text{ 값: } \hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

목적: $\hat{y}$ 과 y의 **차이를 최소화**(정확히는 y축 거리의 제곱의 합)

2.2.2 절편 β_0 의 의미와 함정

절편 β_0 은 $x = 0$ 일 때의 y 추정값이다. 그런데 데이터의 x 범위에 0이 들어 있지 않으면 절편에 큰 의미를 두지 않는 편이 안전하다. 예컨대 cars 데이터에서 speed의 최소값이 4mph이므로, speed = 0에서의 추정값은 외삽(extrapolation)이라 신뢰하기 어렵다.

절편이 음수로 나와도 당황하지 말 것

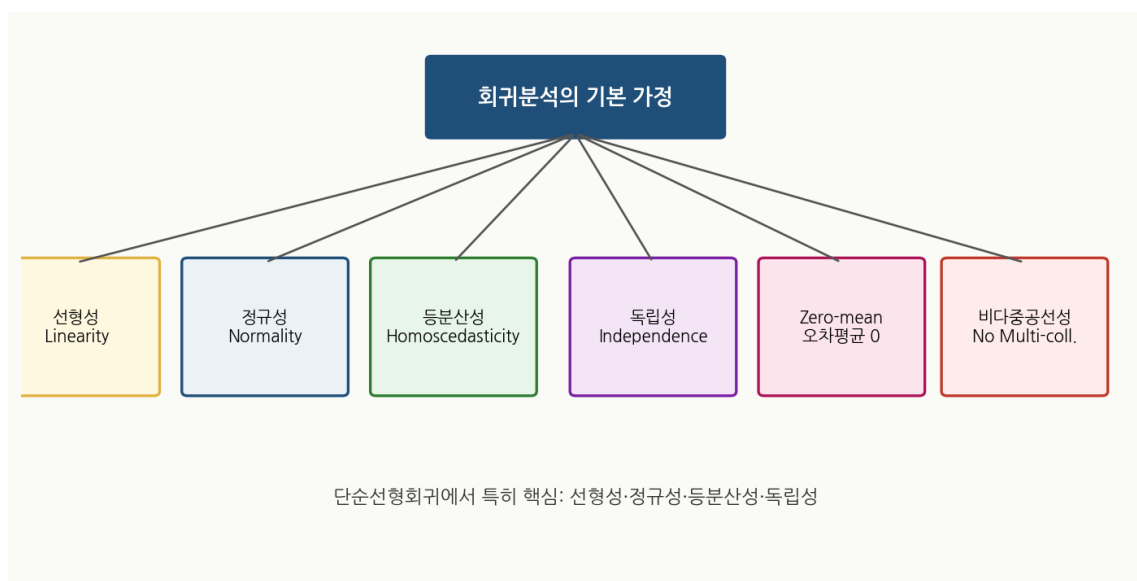
cars 데이터를 회귀하면 절편 $\hat{\beta}_0 = -17.58$ 이 나온다. '속도가 0 이면 제동거리가 음수?'라며 당황할 필요 없다. 데이터의 x 범위 바깥은 회귀분석이 책임지지 않는 영역이다.

절편의 부호보다는 기울기 β_1 과 그 통계적 유의성(p-value)에 집중하자.

2.3 회귀분석의 기본 가정

회귀분석을 '믿을 만한' 결과로 만들기 위해서는 몇 가지 가정이 필요하다. 가정이 깨지면 회귀선은 그려질 수 있지만, 그 결과(특히 p-value와 신뢰구간)는 신뢰할 수 없게 된다. 다음 6가지 가정이 회귀분석의 기둥이다.

2.3.1 선형성·정규성·등분산성·독립성·Zero-mean·다중공선성



[그림 7] 회귀분석의 6 가지 기본 가정.

[가정 6 종] 회귀분석의 전제

- ① 선형성(Linearity): 두 변수의 관계가 직선이다.
- ② 정규성(Normality): 오차항의 분포가 정규분포다.
- ③ 등분산성(Homoscedasticity): 오차항의 분산이 모든 x 값에서 동일하다.
- ④ Zero-conditional mean: 오차항의 기대값(평균)이 0 이다.
- ⑤ 독립성(Independence): 오차항끼리 서로 독립이며 패턴이 없다.
- ⑥ 비다중공선성: 설명변수들이 서로 강한 상관관계를 갖지 않는다(다중회귀 한정).

2.3.2 단순회귀에서 특히 중요한 가정 3 가지

단순선형회귀에서는 설명변수가 하나이므로 ⑥ 다중공선성은 자동으로 해결된다. ④ Zero-mean 도 OLS 가 자동으로 보장한다(잔차의 합이 0 이 되도록 푼다). 그래서 실제로 우리가 잔차분석에서 검토해야 하는 것은 ① 선형성, ② 정규성, ③ 등분산성, ⑤ 독립성, 이 네 가지다. 7 장에서 이를 각각 어떻게 검토하는지 다룬다.

2 장 정리

단순선형회귀의 모형식은 $y = \beta_0 + \beta_1 x + \varepsilon$. 오차 ε 은 이론적, 잔차 $e = y - \hat{y}$ 는 실측 가능.

β_1 의 해석: x 가 1 증가할 때 y 의 평균이 얼마나 변하는가.

단순회귀의 핵심 가정: 선형성·정규성·등분산성·독립성.

3 장. 회귀계수를 찾는 법 — 최소제곱법(OLS)

회귀분석의 핵심 질문은 단 하나다: 어떤 직선이 데이터를 '가장 잘' 설명하는가? 이 질문에 답하려면 먼저 '가장 잘'이라는 말의 뜻을 수학적으로 정의해야 한다. 최소제곱법(Ordinary Least Squares, 이하 OLS)이 200 년 가까이 회귀분석의 표준이 된 이유는 이 정의가 단순하고, 계산이 쉽고, 무엇보다 '닫힌 형태(closed form)'의 해가 있기 때문이다. 본 장에서는 OLS 의 직관과 수식, 그리고 약점까지 다룬다.

3.1 "잘 설명한다"의 정의

그림 위에 점들이 흩어져 있고, 그 점들을 관통하는 직선을 그려야 한다고 하자. 어떤 직선이 '좋은' 직선인가? 직관적으로는 '점들과 직선 사이의 거리가 짧을수록' 좋은 직선이다. 그런데 '거리'에도 종류가 있다 — y 축 방향 거리, x 축 방향 거리, 직선에 수직인 거리. OLS 는 그중 y 축 방향 거리를 사용한다.

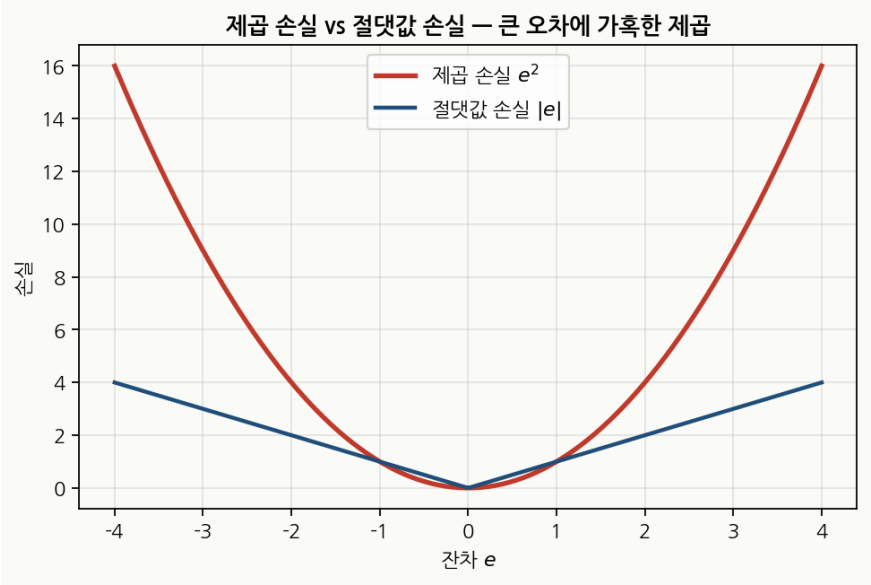
3.1.1 y 축 거리의 제곱합을 왜 최소화하는가

회귀분석의 목적이 ' x 로 y 를 예측'하는 것이기 때문이다. 우리가 예측하고자 하는 것은 y 이고, 예측 오차도 y 방향으로 측정해야 의미가 있다. 그래서 OLS 는 점 (x_i, y_i) 에서 회귀선 $y = \beta_0 + \beta_1 x$ 위의 점 $(x_i, \beta_0 + \beta_1 x_i)$ 까지의 '수직(y 축 방향) 거리'를 잰다. 이 거리가 잔차 $e_i = y_i - \hat{y}_i$ 다.

3.1.2 절댓값이 아니라 제곱인 이유

거리의 합을 최소화하면 되지 굳이 제곱을 할 필요가 있을까? 두 가지 이유가 있다. 첫째, 수학적 편의 — 제곱 함수는 미분이 매끄럽기 때문에 닫힌 형태의 해를 유도하기 쉽다. 절댓값은 0 에서 미분 불가능하다. 둘째, 통계적 의미 — 오차가 정규분포를 따른다고 가정할 때 OLS 는 최

대우도추정(MLE)과 일치한다. 다만 제곱은 큰 오차에 가혹하다는 부작용이 있는데 이는 곧 OLS 의 약점이기도 하다(§3.4).



[그림 8] 제곱 손실과 절댓값 손실의 비교 — 제곱은 큰 오차에 가혹하다.

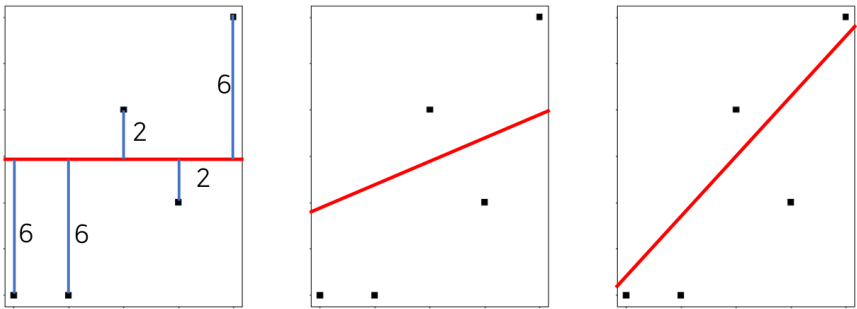
3.2 세 직선의 비교 — OLS 의 직관

같은 다섯 개의 점에 대해 세 개의 후보 직선을 그리고, 각 직선의 잔차제곱합을 계산해 비교한다. 116 에서 79 로, 그리고 79 에서 47 로 줄어드는 모습을 한 단계씩 따라가 보면 OLS 가 무엇을 '최소화'한다는 말인지 손에 잡힐 듯이 이해된다.

 회귀 계수의 추정

 최소제곱법: $(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + (\hat{y}_3 - y_3)^2 + (\hat{y}_4 - y_4)^2 + (\hat{y}_5 - y_5)^2$ 을 최소화

그림 1: $6^2 + 6^2 + 2^2 + 2^2 + 6^2 = 116$



[그림 9] 후보 직선 1 — 잔차제곱합 = $6^2 + 6^2 + 2^2 + 2^2 + 6^2 = 116$.



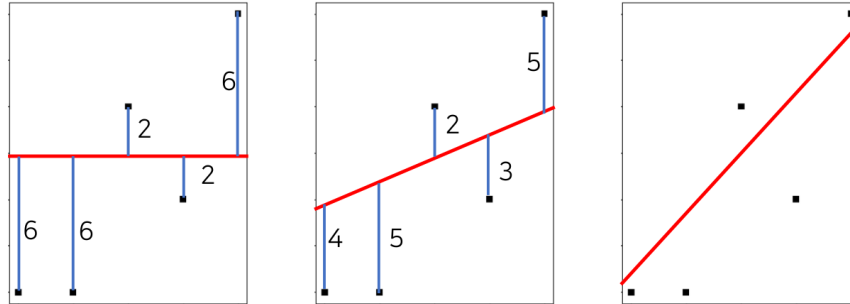
회귀 계수의 추정



최소제곱법: $(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + (\hat{y}_3 - y_3)^2 + (\hat{y}_4 - y_4)^2 + (\hat{y}_5 - y_5)^2$ 을 최소화

그림 1: $6^2 + 6^2 + 2^2 + 2^2 + 6^2 = 116$

그림 2: $4^2 + 5^2 + 2^2 + 3^2 + 5^2 = 79$



[그림 10] 후보 직선 2 — 잔차제곱합 = $4^2 + 5^2 + 2^2 + 3^2 + 5^2 = 79$.



회귀 계수의 추정

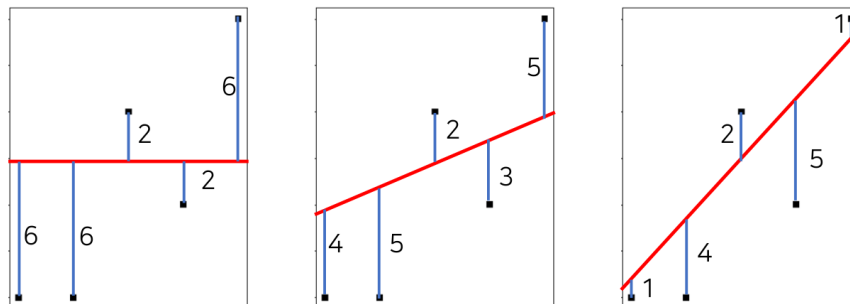


최소제곱법: $(\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + (\hat{y}_3 - y_3)^2 + (\hat{y}_4 - y_4)^2 + (\hat{y}_5 - y_5)^2$ 을 최소화

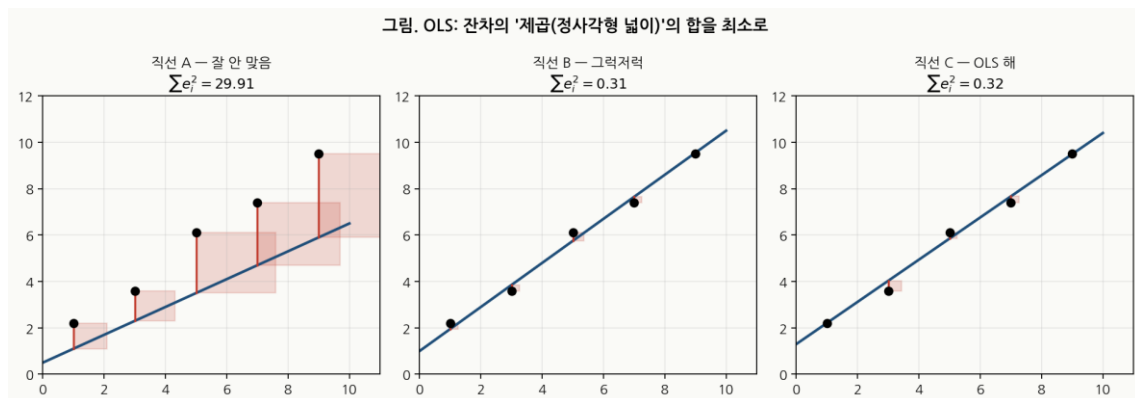
그림 1: $6^2 + 6^2 + 2^2 + 2^2 + 6^2 = 116$

그림 2: $4^2 + 5^2 + 2^2 + 3^2 + 5^2 = 79$

그림 3: $1^2 + 4^2 + 2^2 + 5^2 + 1^2 = 47$



[그림 11] 후보 직선 3 — 잔차제곱합 = $1^2 + 4^2 + 2^2 + 5^2 + 1^2 = 47$. 셋 중 가장 좋다.



[그림 12] OLS의 의미를 '잔차 제곱(정사각형 넓이)'으로 시각화. 빨간 사각형들의 합이 작을수록 좋은 직선이다.

[OLS의 정의] 한 줄로 요약

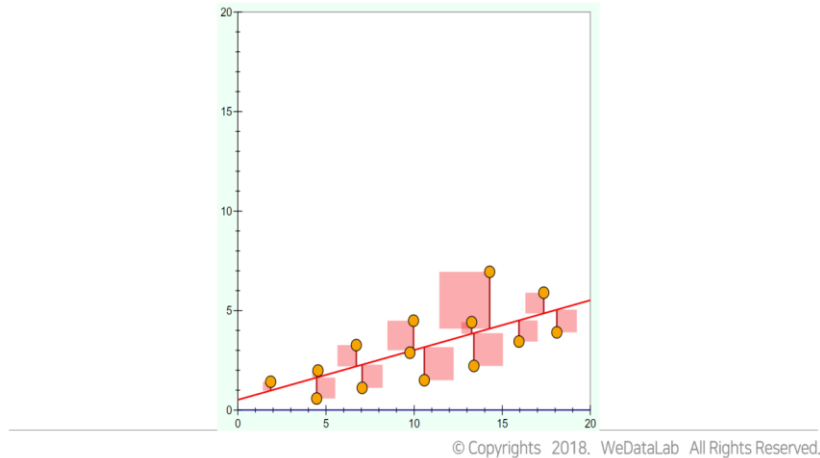
데이터 $(x_1, y_1), \dots, (x_n, y_n)$ 이 주어졌을 때, OLS는 다음 최적화 문제를 푼다:

$$\min \sum_i (y_i - (\beta_0 + \beta_1 x_i))^2 \quad (\beta_0, \beta_1 \in \mathbb{R})$$

즉, 잔차의 '제곱합'을 최소화하도록 β_0, β_1 을 추정한다.

잔차제곱합을 시각적으로 표현

$$Cost(\hat{\beta}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2$$



[그림 13] 잔차제곱합의 시각화 — $Cost(\hat{\beta}) = \sum (\hat{y}_i - y_i)^2$.

3.3 OLS 수식 유도

3.3.1 일변수 closed-form 해

OLS의 강점은 해를 한 번에 닫힌 형태로 구할 수 있다는 점이다. 손실함수 $S(\beta_0, \beta_1) = \sum (y_i - \beta_0 - \beta_1 x_i)^2$ 를 β_0 과 β_1 에 대해 각각 편미분하여 0으로 두면 두 개의 정규방정식 (normal equations)이 나오고, 이를 풀면 다음과 같다.

[OLS closed-form] 단순회귀의 해

$$\beta_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$$

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

β_1 은 x 와 y 의 공분산을 x 의 분산으로 나눈 값과 같다.

3.3.2 행렬 표기와 $\hat{\beta} = (X^T X)^{-1} X^T y$

다중회귀로 확장하면 행렬 표기가 훨씬 깔끔하다. 설명변수가 여러 개라도 닫힌 형태의 해가 그대로 살아남는다.

[행렬 표기] 다변수 OLS

$$\min \sum (y_i - (\beta_0 + \beta_1 x_i))^2 \Leftrightarrow \min (y - X\beta)^T (y - X\beta)$$

$$y = [y_1, \dots, y_n]^T, \quad X = [1, x_1; 1, x_2; \dots; 1, x_n], \quad \beta = [\beta_0, \beta_1]^T$$

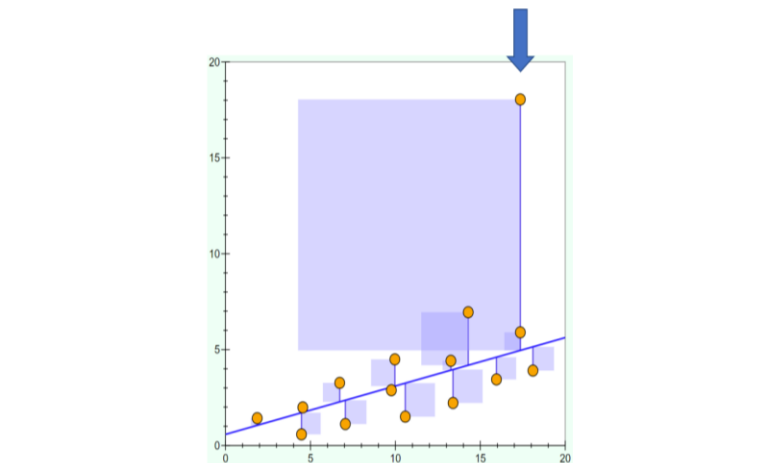
위 최적화 문제의 해: $\beta = (X^T X)^{-1} X^T y$

3.4 OLS의 약점 — 이상치에 약하다

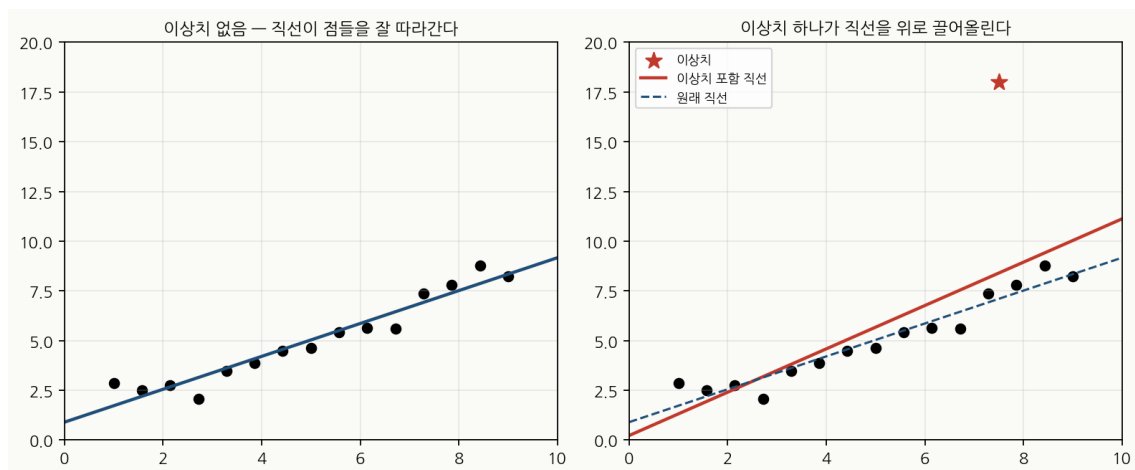
3.4.1 제곱오차가 큰 오차에 가혹한 이유

잔차가 1 일 때 손실은 1, 잔차가 10 일 때 손실은 100 이 된다. 큰 오차 하나가 작은 오차 100 개와 같은 비중을 갖는 것이다. 이것이 OLS가 이상치에 약한 본질적인 이유다. 데이터에 큰 이상치 하나가 끼어 있으면 OLS는 그 이상치를 줄이려고 직선 전체를 기울인다.

제곱오차를 사용하면 오차가 커질수록 비용이 엄청나게 커진다.



3.4.2 이상치가 직선을 끌어당기는 시각화



[그림 15] 이상치 하나가 회귀선을 위로 끌어올린다. 점선이 원래 직선, 빨간선이 이상치 포함.

3 장 정리

OLS 는 'y 축 거리 제곱합'의 최소화로 회귀계수를 정의한다.

단순회귀의 해: $\beta_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$, $\beta_0 = \bar{y} - \beta_1 \bar{x}$.

행렬 표기: $\beta = (X^T X)^{-1} X^T y$ — 다중회귀로도 그대로 확장된다.

약점: 이상치에 약하다. 큰 잔차 하나가 직선 체를 끌어당긴다.

4 장. 회귀식 해석하기

3 장까지는 회귀선을 '어떻게 구하는가'에 집중했다. 이제 실제로 구해 놓은 회귀선의 결과표를 '어떻게 읽는가'를 다룰 차례다. sklearn 의 LinearRegression 함수 출력은 회귀계수표 (Coefficients) 형태로 나오며, 거기에 적힌 Estimate, Std. Error, Pr(>|t|) 라는 세 열의 의미를 알아야 비로소 회귀분석을 '읽었다'고 할 수 있다.

4.1 cars 데이터 소개

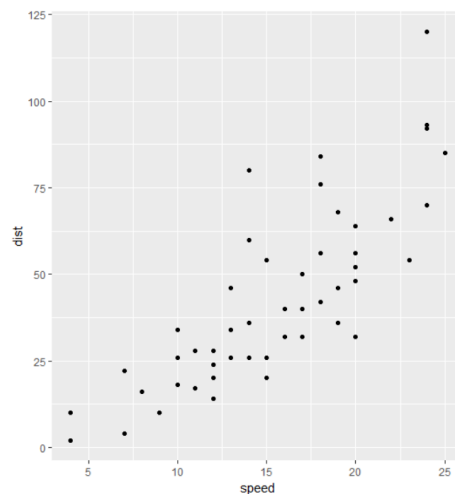
4.1.1 speed 와 dist — 자동차 제동거리 데이터 (n=50)

cars 는 1920 년대 미국에서 자동차의 주행 속도(speed, mph)에 따른 제동거리(dist, ft)를 50 건 측정한 자료다. 직관적으로 속도가 빠를수록 제동거리가 길어질 것이라 기대된다.



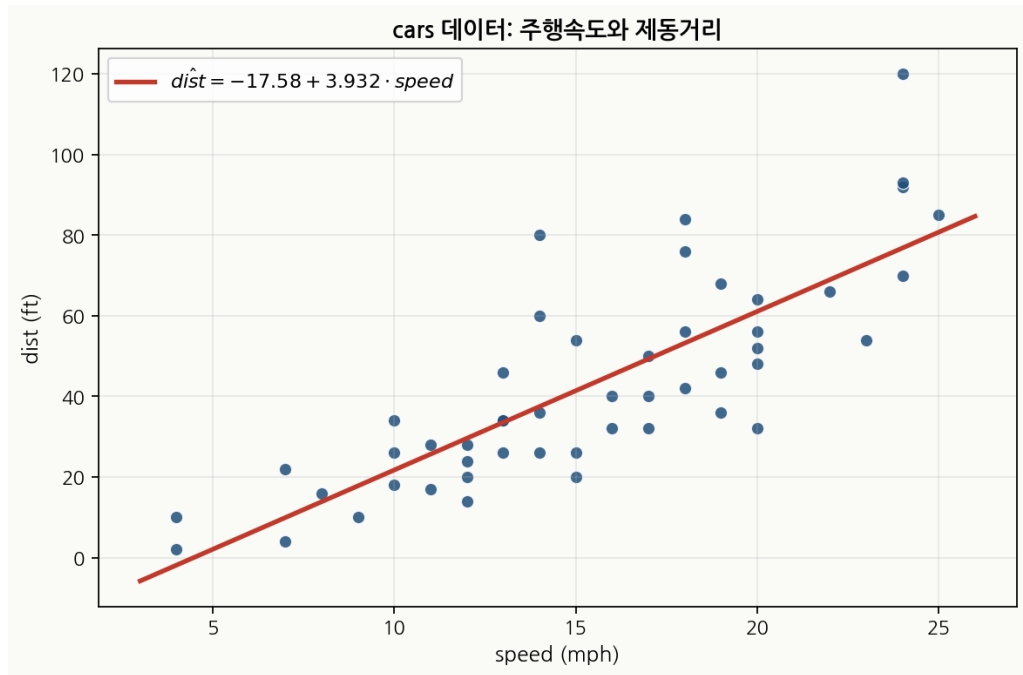
예시: "cars" 데이터(내장 데이터) : 브레이크가 작동되는 순간의 자동차의 주행 속도(Speed)에 따른 자동차 제동 거리(StopDist)를 조사한 자료

	speed	dist
1	4	2
2	4	10
3	7	4
4	7	22
5	8	16
6	9	10
7	10	18
8	10	26
9	10	34
10	11	17
11	11	28
12	12	14
13	12	20
14	12	24
15	12	28
16	13	26
17	13	34
18	13	34
19	13	46
20	14	26
21	14	36
22	14	60



[그림 16] cars 데이터 — 주행속도와 제동거리 50 건 .

4.1.2 산점도로 확인하는 양의 상관



[그림 17] cars 데이터 산점도와 회귀선. 양의 선형 추세가 뚜렷하다.

4.2 회귀계수표(Coefficients) 읽는 법

회귀계수표는 다음과 같이 생겼다. 각 열의 의미를 차례로 살펴보자.



예시: "cars" 데이터(내장 데이터)

반응변수를 'dist'로, 'speed'를 설명변수로 모형을 적합시킨 결과

Coefficients	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-17.5791	6.7584	-2.601	0.0123
speed	3.9324	0.4155	9.464	1.49e-12

[그림 18] cars 데이터에 대한 회귀계수표 — Estimate, Std.Error, t value, Pr(>|t|).

[열 의미] 회귀계수표의 네 열

Estimate (β) — OLS로 추정한 회귀계수 값

Std. Error (SE) — 추정값의 표준오차. 표본을 바꿔 가며 추정했을 때 β 의 변동성

t value — β / SE . 회귀계수가 0과 얼마나 떨어져 있는지를 SE 단위로 잴 값

Pr(>|t|) — p-value. $\beta = 0$ 이라는 귀무가설이 옳을 확률(직관적 표현)

4.2.1 추정된 회귀식 $\text{dist} = -17.58 + 3.93 \times \text{speed}$ 의 의미

표에서 (Intercept) 행의 Estimate 가 -17.5791 이고, speed 행의 Estimate 가 3.9324 다. 이를 회귀식으로 쓰면 다음과 같다.

[적합된 회귀식]

$$\text{dist} = -17.5791 + 3.9324 \times \text{speed}$$

해석: 주행속도가 1mph 늘어날 때 제동거리는 평균 3.93ft 늘어난다.

절편 -17.58 은 speed=0 에서의 외삽이므로 실용적 의미를 두지 않는다.

4.3 통계적 유의성과 p-value

4.3.1 귀무가설 $H_0: \beta = 0$ 과 대립가설 $H_1: \beta \neq 0$

회귀계수의 p-value 가 작다는 것은 ' $\beta = 0$ 이라는 귀무가설이 사실일 때, 우리가 본 데이터가 나올 확률이 매우 낮다'는 뜻이다. 직관적으로는 ' β 가 0 이라기에는 데이터가 너무 또렷한 관계를 보여 준다'고 읽으면 된다.

cars 데이터에서 speed 행의 p-value 는 1.49e-12 로 거의 0 에 가깝다. 즉 '속도와 제동거리가 선형 관계가 없다'는 가설은 통계적으로 매우 강하게 기각된다. 따라서 회귀계수 $\hat{\beta}_1 = 3.93$ 은 통계적으로 유의(significant)하다고 결론 내릴 수 있다.

4.3.2 유의수준 0.05 의 관습

관습적으로 p-value 가 0.05 보다 작으면 '유의하다'고 본다. 0.05 라는 숫자에는 깊은 수학적 의미가 있는 것이 아니라, 단지 학계의 합의일 뿐이다. 분야에 따라 0.01 을 쓰기도 하고, 의학·물리에서는 더 엄격한 기준을 쓰기도 한다.

함정 · p-value 는 효과의 '크기'가 아니다

p-value 가 작다고 효과가 크다는 뜻이 아니다. p-value 는 '이 효과가 우연이 아닐 확률(직관적 표현)'을 말할 뿐, 효과의 크기는 Estimate 값이다.

효과 크기는 β 의 값과 도메인 의미로, **통계적 유의성**은 p-value 로 따로 판단해야 한다.

4장 정리

Estimate 는 β , Std. Error 는 그 표본 변동성, t value = β/SE , $\Pr(|t|)$ 가 p-value.

cars 데이터: $\text{dist} = -17.58 + 3.93 \times \text{speed}$.

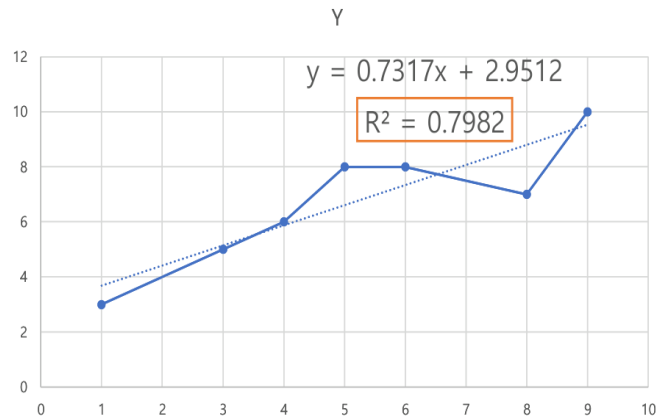
p-value 가 0.05 보다 작으면 '0 이 아니라고 통계적으로 결론 내릴 만하다'.

p-value 의 크기 $\neq$ 효과의 크기. 둘은 다른 차원이다.

5 장. 모형의 적합도 평가 — 결정계수 R^2

회귀선을 구하고, 계수를 해석하는 것까지 마쳤다고 해도 한 가지 질문이 남는다: '이 회귀선이 데이터를 얼마나 잘 설명하는가?' 같은 데이터에 대해서도 회귀선이 어떤 데이터셋에서는 거의 완벽하게 들어맞고, 다른 데이터셋에서는 형편없을 수 있다. 이를 정량화하는 도구가 결정계수 (coefficient of determination) R^2 이다.

5.1 직선이 데이터를 "설명한다"는 것



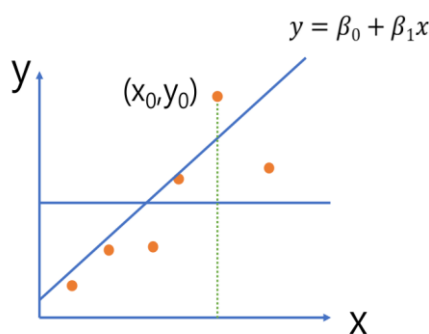
- R-제곱 값: 직선이 데이터를 설명하는 정도, 상관계수와 관련된 값
- R-제곱 값의 정확한 의미는? 그리고 구하는 방법은?

[그림 25] R^2 의 의미 — 직선이 데이터를 설명하는 정도.

5.1.1 한 점 (x_0, y_0) 을 두고 보는 두 가지 거리

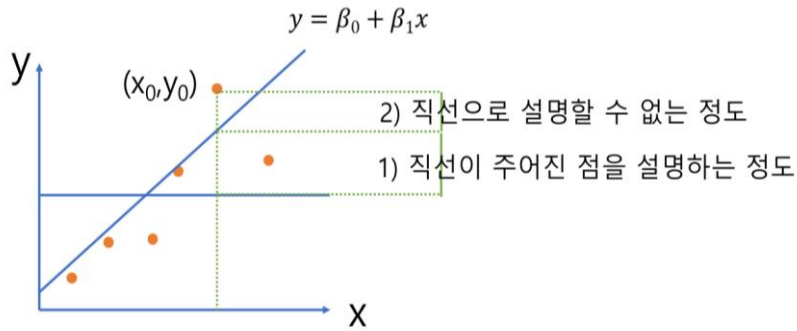
한 점 (x_0, y_0) 을 두고 회귀선을 그렸다고 하자. 이 점에서 '평균선 $\bar{y}$ '까지의 수직 거리는 그 점이 전체 평균에서 얼마나 떨어져 있는지를 보여준다. 이 거리 중 일부는 회귀선이 '설명'하고, 일부는 '설명하지 못하고' 잔차로 남는다.

따라서 어떤 점 (x_0, y_0) 에 대해 $\bar{y}$ 와 $\beta_0 + \beta_1 x_0$ 의 차이를 고려한다면, 직선 $y = \beta_0 + \beta_1 x$ 가 주어진 점 (x_0, y_0) 를 얼마나 잘 설명하고 있는지 알 수 있다.



[그림 19] 한 점과 회귀선, 그리고 평균선 $\bar{y}$.

여기서 실제 값 y_0 와 $\beta_0 + \beta_1 x_0$ 는 값이 다를 수 있다(2) 표시).
 이러한 차이는 직선으로 설명할 수 없는, 오차 ε 에 해당하는 부분이다.



직선이 설명하는 정도 = $\hat{y}_0 - \bar{y}$
 직선으로 설명할 수 없는 정도 = $y_0 - \hat{y}_0$ (잔차).

5.1.2 설명되는 부분 vs 설명되지 않는 부분

결국 한 점의 변동성($y_0 - \bar{y}$)은 두 조각으로 쪼개진다.

[변동성 분해] 한 점에서의 항등식

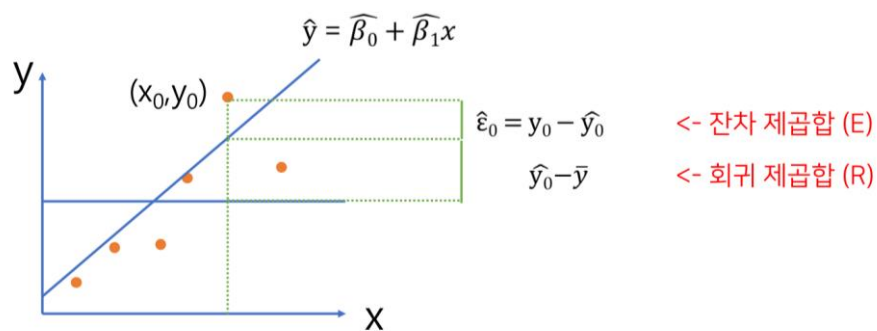
$$y_0 - \bar{y} = (\hat{y}_0 - \bar{y}) + (y_0 - \hat{y}_0)$$

↑ ↑

설명되는 부분 설명되지 않는 부분(잔차)

$$y_0 - \bar{y} = \hat{y}_0 - \bar{y} + \hat{\varepsilon}_0$$

(전체 변동성) = (직선으로 설명되는 변동성) + (직선으로 설명되지 않는 변동성)



[그림 22] 잔차제곱합(E)과 회귀제곱합(R)의 시각화 .

5.2 제곱합 분해: $SST = SSR + SSE$

위의 변동성 분해를 모든 점에 대해 합치면 다음의 핵심 항등식이 나온다.

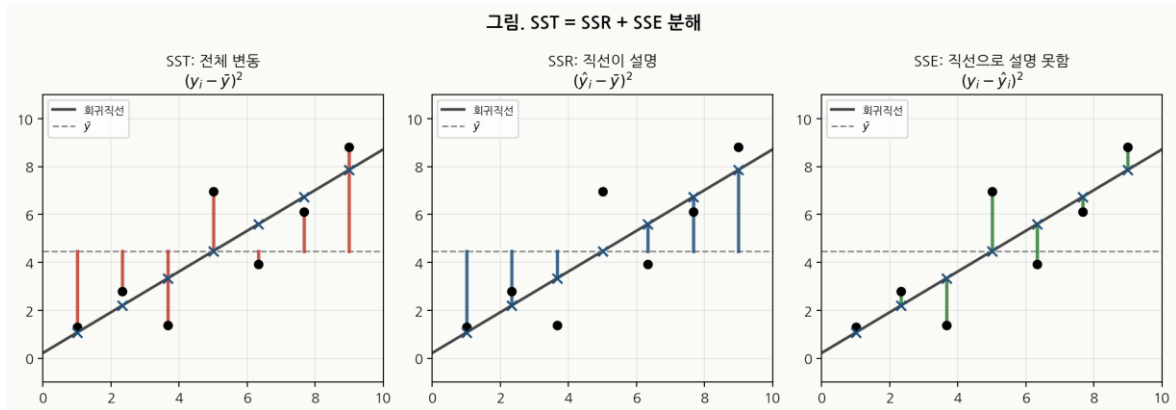
[제곱합 분해 $SST = SSR + SSE$]

$SST = \sum (y_j - \bar{y})^2$ (Total Sum of Squares — 전체 변동성)

$SSR = \sum (\hat{y}_j - \bar{y})^2$ (Regression — 회귀직선이 설명하는 변동성)

$SSE = \sum (y_j - \hat{y}_j)^2$ (Error — 회귀직선이 설명 못하는 변동성)

$SST = SSR + SSE$ (이 등식이 항상 성립한다)



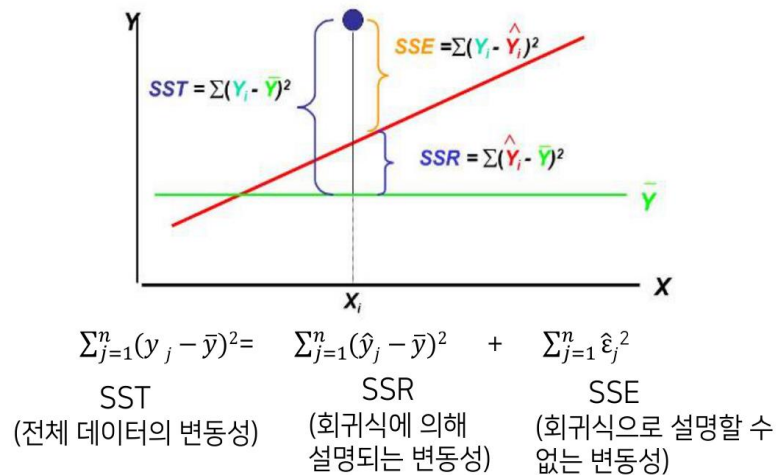
[그림 23] $SST = SSR + SSE$ 분해 — 빨강은 평균에서의 거리, 파랑은 회귀선이 설명하는 거리, 초록은 잔차.



SST: 전체 데이터의 변동성(y 값이 평균과 떨어져있는 정도)

SSR: 회귀직선으로 설명되는 변동성

SSE: 회귀직선으로 설명되지 않는 변동성($\hat{\epsilon}_j = \hat{y}_j - y_j$; 잔차)



[그림 24] $SST \cdot SSR \cdot SSE$ 의 의미 .

5.2.1 $SST \cdot SSR \cdot SSE$ 의 기하학적 의미

SST는 데이터 전체가 평균 $\bar{y}$ 로부터 얼마나 흩어져 있는지의 총량이다. 만약 회귀선을 그리지 않고 모든 예측을 $\bar{y}$ 로 한다면 손실은 SST가 된다. SSR은 회귀선을 도입함으로써 줄어든 변동성의 양 — 즉 회귀선이 '벌어들인' 부분이고, SSE는 그래도 남은 잔차의 양이다.

5.3 결정계수 R^2 의 정의와 해석

[정의] 결정계수 R^2

$$R^2 = SSR / SST = 1 - SSE / SST$$

$$0 \leq R^2 \leq 1 \quad (\because SST = SSR + SSE)$$

R^2 가 1에 가까울수록 회귀선이 데이터를 잘 설명한다.

5.3.1 $R^2 = 0$ 과 $R^2 = 1$ 의 의미

$R^2 = 1$ 이라면 모든 점이 회귀선 위에 완벽하게 놓여 있다는 뜻이다(잔차가 모두 0). 반대로 $R^2 = 0$ 이라면 회귀선이 $\bar{y}$ 직선과 같다는 뜻 — 회귀를 한 보람이 전혀 없다. 실제 데이터에서는 그 중간 어디쯤이며, cars 데이터의 R^2 는 약 0.6511로, '제동거리 변동의 약 65%를 속도로 설명할 수 있다'고 해석한다.

5.3.2 단순회귀에서 $R^2 = r^2$ 인 이유

단순선형회귀에 한해, 결정계수 R^2 은 피어슨 상관계수 r 의 제곱과 같다. 즉 $R^2 = r^2$ 이다. cars 데이터의 경우 상관계수 $r \approx 0.8069$ 이고 $R^2 \approx 0.6511$ 인데, $0.8069^2 = 0.651$ 이다. 이 멋진 관계는 단순회귀에서만 성립한다. 다중회귀에서는 R^2 가 더 일반적인 의미를 갖는다.

R^2 만으로 모형을 판단하지 말 것

R^2 가 높다고 반드시 좋은 모형은 아니다. 예를 들어 데이터에 강한 곡선 패턴이 있는데도 직선으로 적합하면 R^2 가 그럭저럭 나올 수 있다 — 하지만 잔차도를 그려 보면 곡선 패턴이 보인다. R^2 와 잔차분석은 반드시 함께 봐야 한다 (7 장).

5 장 정리

한 점의 변동성: $(y - \bar{y}) = (\hat{y} - \bar{y}) + (y - \hat{y})$.

전체 합으로: $SST = SSR + SSE$. 이 등식은 항상 성립.

결정계수 $R^2 = SSR/SST = 1 - SSE/SST$. 0~1 사이.

단순회귀에서는 $R^2 = r^2$ (피어슨 상관의 제곱)이 성립.

6 장. Python 으로 실습하는 단순선형회귀

이제 실습으로 가보자. scikit-learn 에는 선형회귀가 한 줄의 명령으로 끝나도록 만들어 둔 LinearRegression 클래스가 있다. 본 장은 cars 데이터를 가지고 데이터 살펴보기 → 상관분석 → 모형 적합 → 결과 해석 → 예측의 다섯 단계를 차례로 따라간다.

6.1 데이터 살펴보기

6.1.1 cars 데이터 로드와 요약 통계

데이터의 처음 몇 행과 요약 통계를 본 뒤 산점도를 그려 보는 것이 첫 단계다.

```
# 데이터 로드

# cars 데이터 - speed(mph) vs dist(ft), 50 개 관측치
import pandas as pd

data = pd.DataFrame({
    'speed': [4,4,7,7,8,9,10,10,10,11,11,12,12,12,12,13,13,13,13,
              14,14,14,14,15,15,15,16,16,17,17,17,18,18,18,18,19,19,19,
              20,20,20,20,20,22,23,24,24,24,24,25],
    'dist': [2,10,4,22,16,10,18,26,34,17,28,14,20,24,28,26,34,34,46,
             26,36,60,80,20,26,54,32,40,32,40,50,42,56,76,84,36,46,68,
             32,48,52,56,64,66,54,70,92,93,120,85]
})

print(data.describe())
print(f"speed-dist 상관계수: {data['speed'].corr(data['dist']):.4f}")
```

6.1.2 matplotlib/seaborn 로 그리는 산점도

```
# matplotlib + seaborn 으로 산점도 + 회귀선
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(8, 5))
plt.scatter(data['speed'], data['dist'], alpha=0.6, edgecolor='k', label='관찰값')

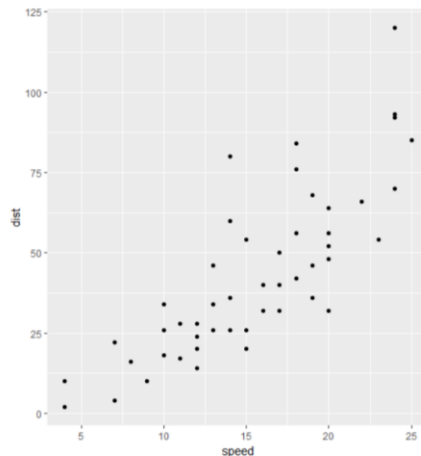
# 또는 seaborn 으로 같은 그림을 한 줄에
# import seaborn as sns
# sns.regplot(data=data, x='speed', y='dist'); plt.show()

# 회귀선 그리기 - 학습된 model 이 있을 경우에 사용
x_line = np.linspace(data['speed'].min(), data['speed'].max(), 100).reshape(-1, 1)
plt.plot(x_line, model.predict(x_line), color='red', linewidth=2, label='회귀선')
plt.xlabel('speed (mph)'); plt.ylabel('dist (ft)')
plt.title('cars 데이터의 단순선형회귀')
plt.legend(); plt.grid(True, alpha=0.3); plt.tight_layout()
plt.show()
```



데이터 살펴보기

```
library(ggplot2)
ggplot(data=data1, aes(x=speed, y=dist)) + geom_point()
```



산점도를 확인한 결과 둘 사이에는 양의 상관관계가(선형적 연관성이) 존재하는 것으로 짐작할 수 있다.

-> 상관분석을 통해 상관관계의 유무를 확인해보자.

[그림 26] matplotlib/seaborn 로 그린 cars 산점도 — 양의 상관 추세가 뚜렷하다.

6.2 상관분석 먼저

회귀분석 전에 두 변수 사이에 '직선적' 관계가 있긴 한지 상관분석으로 미리 확인하는 것이 정석이다. `cor()`는 상관계수를, `cor.test()`는 그 통계적 유의성까지 함께 알려준다.

6.2.1 `corr()` - 상관계수 먼저보기

```
# 1. 두 변수 간의 상관계수만 구하기
print(cars['speed'].corr(cars['dist']))
# 출력: 0.8068949006410302

# 2. 데이터프레임 전체의 상관계수 행렬(Matrix) 구하기
print(cars.corr())

# 출력:
#           speed      dist
# speed  1.000000  0.806895
# dist   0.806895  1.000000
```

6.2.2 heatmap으로 상관행렬 시각화

```
plt.figure(figsize=(6, 5))
M = cars.corr()
sns.heatmap(M, annot=True, cmap='RdBu_r', vmin=-1, vmax=1, square=True)
plt.title("Correlation Matrix - Cars", fontsize=14)
plt.show()
```

[그림 28] corrplot 으로 본 상관행렬.

6.3 sklearn 으로 모형 적합

6.3.1 lm(y ~ x, data=) 의 기본 사용법

sklearn 의 LinearRegression 은 'Linear Model'의 약자다. 회귀분석뿐만 아니라 분산분석(ANOVA)·공분산분석(ANCOVA)까지 같은 함수 하나로 다룬다. 단순선형회귀는 $lm(y \sim x, data=df)$ 가 끝이다.

```
# 변수 분리 (X 는 2 차원 데이터프레임 구조여야 합니다)
X = cars[['speed']]
y = cars['dist']

# 2. 선형 회귀 모델 생성 및 학습 (lm()과 동일)
model = LinearRegression()
model.fit(X, y)

# 3. 결과 확인 (R 의 fit 결과에 대응하는 값들)
print("Call:")
print(f"LinearRegression() 모델 학습 완료 (Formula: dist ~ speed)\n")
print("Coefficients:")
print(f"Intercept (절편): {model.intercept_}")
print(f"speed (기울기): {model.coef_[0]}")
```



모형 적합

```
> fit.cars <- lm(dist ~ speed, data=cars)
> fit.cars

Call:
lm(formula = dist ~ speed, data = cars)

Coefficients:
(Intercept)      speed
    -17.579         3.932

> summary(fit.cars )
```

회귀모형은 $lm(y \sim x)$ 함수를 이용한다. 여기서 lm은 선형 모형(Linear Model)의 약자로, 회귀분석, 분산분석 등이 대표적인 선형모형이다.

정답은 $y = -17.579 + 3.932 \times x$ 였던 것이었다!

[그림 29] sklearn LinearRegression 적합 결과 — $dist = -17.579 + 3.932 \times speed$.

6.3.2 statsmodels.summary() 가 보여 주는 3 가지 정보

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

model_sm = smf.ols('dist ~ speed', data=cars).fit()

# statsmodels 에서 제공하는 summary
print(model_sm.summary())

# 개별 통계량 접근
```



```
print(f'R-squared: {model_sm.rsquared:.4f}')
print(f'Adj. R-squared: {model_sm.rsquared_adj:.4f}')
print(f'F-statistic: {model_sm.fvalue:.4f}')
print(f'Prob (F-statistic): {model_sm.f_pvalue:.4e}')
```

statsmodels OLS 의 결과 객체에 .summary() 를 호출하면 ① 잔차 분포 요약, ② 회귀계수표 (coef·std err·t·P>|t|·신뢰구간), ③ 모형 전체의 적합도(R^2 ·Adj. R^2 ·F-통계량·전체 p-value) 까지 한꺼번에 알려준다.



Summary() 는 3가지 정보를 보여준다

```
> fit.cars<-lm(dist~speed, data=cars)
> summary(fit.cars)
```

Call:

```
lm(formula = dist ~ speed, data = cars)
```

Residuals:

Min	1Q	Median	3Q	Max
-29.069	-9.525	-2.272	9.215	43.201

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-17.5791	6.7584	-2.601	0.0123 *
speed	3.9324	0.4155	9.464	1.49e-12 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 15.38 on 48 degrees of freedom
Multiple R-squared: 0.6511, Adjusted R-squared: 0.6438
F-statistic: 89.57 on 1 and 48 DF, p-value: 1.49e-12

[그림 30] statsmodels .summary() 가 보여 주는 3 가지 정보 .

6.4 lm 객체 파헤치기

6.4.1 coefficients / fitted.values / residuals

statsmodels 의 OLSResults 객체는 다양한 속성을 가진다. 자주 쓰는 것은 .params(회귀계수), .fittedvalues(적합값), .resid(잔차), .df_resid(자유도), .rsquared(R^2), .conf_int(신뢰구간) 정도다. sklearn LinearRegression 도 .coef_, .intercept_, .score() 로 비슷한 정보를 준다.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. 데이터 준비 및 모델 학습
cars = sm.datasets.get_rdataset("cars", "datasets").data
fit = smf.ols('dist ~ speed', data=cars).fit()

# 2. 회귀계수 확인
print("--- 회귀계수 ---")
print(fit.params)
```

```
# 3. 적합값(예측값) 앞부분
print("--- 적합값 (앞의 6 개) ---")
print(fit.fittedvalues.head())

# 4. 잔차 앞부분 확인
print("--- 잔차 (앞의 6 개) ---")
print(fit.resid.head())
```

fitted.values 는 회귀식에 대입한 적합값이다.
residuals 는 $y - \hat{y}$ 의 값이다.

6.4.2 model / call / df.residual 등

```
fit = smf.ols('dist ~ speed', data=cars).fit()

# 1. 잔차 자유도 (R 의 fit$df.residual)
print("--- 잔차 자유도 ---")
print(fit.df_resid)

# 2. 어떻게 호출되었는지 공식 확인
print("--- 호출 공식(Formula) ---")
print(fit.model.formula)

# 3. 모형에 들어간 데이터 앞부분 확인
print("--- 모형에 사용된 데이터 (앞의 2 개) ---")
print(fit.model.data.frame.head(2))
```

6.5 predict() 로 예측하기

6.5.1 새 데이터에 대한 예측값

회귀모형은 결국 새 데이터에 대한 예측에 쓰이기 위해 만드는 도구다. sklearn 에서는 model.predict(new_X), statsmodels 에서는 results.predict(sm.add_constant(new_X)) 가 새 X 를 받아 예측값을 돌려준다. statsmodels 의 .get_prediction() 은 신뢰구간·예측구간도 함께 준다.

```
fit = smf.ols('dist ~ speed', data=cars).fit()

# 2. 새로운 데이터프레임 생성
new_data = pd.DataFrame({'speed': [10, 20, 30]})

# 3. 예측하기
predictions = fit.predict(new_data)
print(predictions)
```

[그림 34] predict() 사용 (Python sklearn/statsmodels) .

6.5.2 신뢰구간과 예측구간

interval = "confidence"는 회귀선 자체의 신뢰구간(평균 반응의 구간)을, interval = "prediction"은 한 개별 관측값에 대한 예측구간을 돌려준다. 예측구간이 항상 신뢰구간보다 넓다 — 개별 관측값에는 자체 변동성(잔차)이 더 끼기 때문이다.

```
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. 모델 학습 및 새 데이터 준비
cars = sm.datasets.get_rdataset("cars", "datasets").data
fit = smf.ols('dist ~ speed', data=cars).fit()
new_data = pd.DataFrame({'speed': [10, 20, 30]})

# 2. 신뢰구간 (Confidence Interval) 구하기
pred_ci = fit.get_prediction(new_data).summary_frame(alpha=0.05)
print("--- 95% 신뢰구간 (Confidence Interval) ---")
print(pred_ci[['mean', 'mean_ci_lower', 'mean_ci_upper']])
print("\n" + "="*50 + "\n")

# 3. 예측구간 (Prediction Interval) 구하기
# 결과 객체는 같으므로 출력할 컬럼만 obs_ci(관측치 구간)로 변경합니다.
print("--- 95% 예측구간 (Prediction Interval) ---")
print(pred_ci[['mean', 'obs_ci_lower', 'obs_ci_upper']])
```

[그림 35] predict() 결과 (Python) .

팁 · 신뢰구간 vs 예측구간

신뢰구간: 'speed=20 일 때 dist 의 평균은 어디?'

예측구간: 'speed=20 인 새 차 한 대의 dist 는 어디 있을까?'

후자가 항상 더 넓다.

6장 정리

lm(y ~ x, data=df)로 단순회귀 적합, summary()로 한꺼번에 진단.

lm 객체에는 coefficients, fitted.values, residuals 등 12 개 슬롯이 있다.

predict(fit, newdata=...)로 예측, interval='confidence'와 'prediction'을 구분.

7장. 잔차분석 — 가정 검토

회귀식을 만들었다고 끝이 아니다. 4장에서 본 p-value와 5장의 R^2 은 모두 '회귀분석의 가정이 맞다'는 전제 하에 의미를 갖는다. 가정이 깨졌는데도 그 결과를 그대로 믿는 것은 토대 없는 건물에 살라는 말과 같다. 잔차분석은 이 가정을 데이터로 검토하는 과정이다.

7.1 왜 잔차를 봐야 하는가

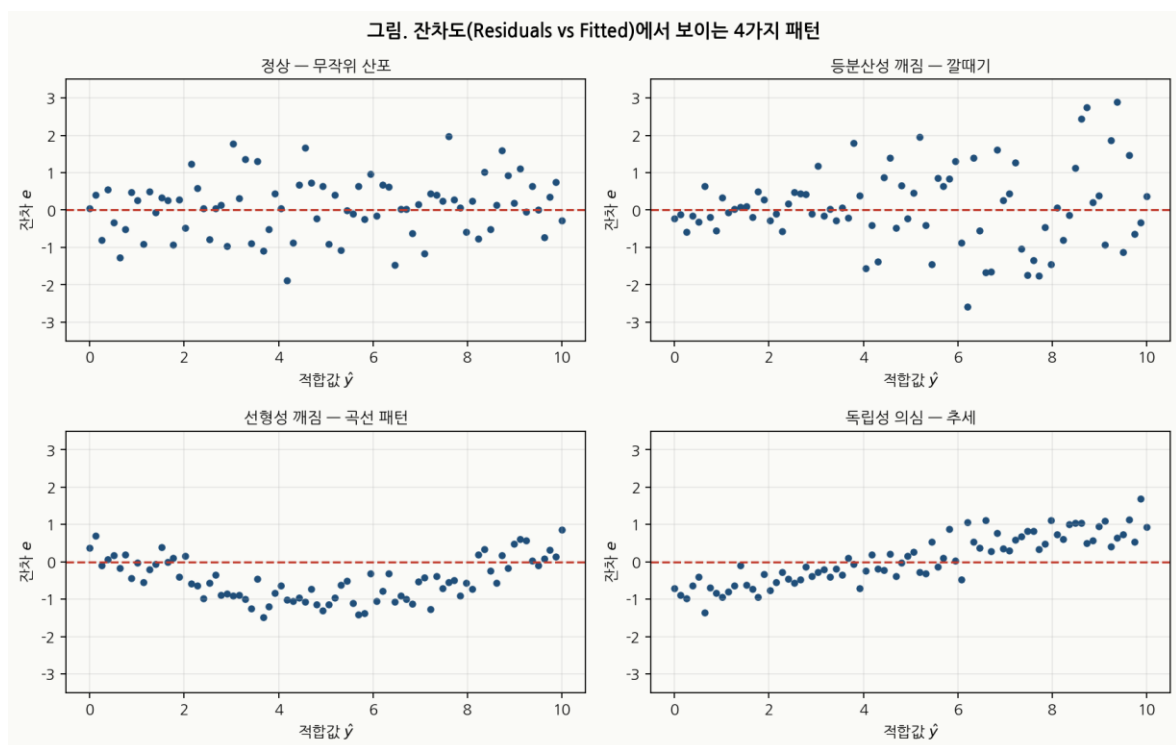
7.1.1 가정이 깨지면 무엇이 잘못되는가

잔차의 분산이 들쭉날쭉(등분산성 위배)하면 회귀계수의 표준오차 추정이 어긋나, p-value와 신뢰구간이 거짓이 된다. 잔차가 정규분포가 아니면(정규성 위배) 작은 표본에서 t-검정의 신뢰성이 흔들린다. 잔차에 패턴이 보이면(선형성 위배) 직선 모형 자체가 잘못 가정된 것이라 회귀선의 모양 자체를 바꿔야 한다.

7.1.2 잔차가 0 주변에서 무작위로 흩어져야 한다

이상적인 잔차는 오차가 무작위로 생성된 것이므로, 0 주변에서 어떤 패턴 없이 음양의 값으로 흩어져 있어야 한다. 잔차도(Residuals vs Fitted)에서 빨간 추세선이 0 선에 가깝게 평평하면 OK이고, 깔때기·곡선·계단 등이 보이면 가정 위배 신호다.

7.2 잔차도 (Residuals vs Fitted)



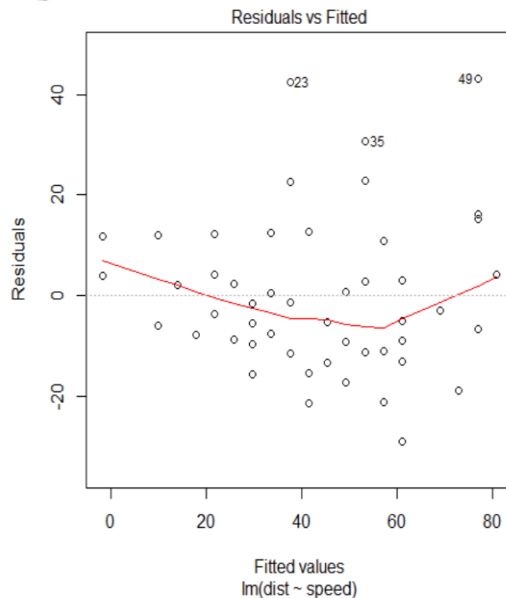
[그림 36] 잔차도에서 나타나는 4가지 패턴 — 정상 / 깔때기 / 곡선 / 추세.

7.2.1 패턴이 없으면 OK, 깔때기 모양이면 등분산성 의심

왼쪽 위의 '정상' 그림은 잔차가 0 주변에서 무작위로 흩어진 모습이다. 오른쪽 위는 적합값이 커질수록 잔차의 분산이 커지는 깔때기(funnel) 모양 — 등분산성이 깨진 전형적 사례다. 왼쪽 아래는 곡선 패턴 — 선형성이 깨졌다는 신호이며, 변환이나 다항회귀가 필요하다. 오른쪽 아래는 추세 — 독립성/Zero-mean 가정에 의심이 든다.



잔차분석



`> plot(fit.cars, 1)`

잔차도를 확인해보니 적합값 (Fitted values)이 커질수록 잔차가 넓게 퍼지는 경향이 보인다.
(등분산성 가정 검토 필요)

<참고> 잔차도에서 빨간 직선이 0에 가까울수록 가정을 잘 만족한다고 볼 수 있다.

© Copyrights 2018. WeDataLab All Rights Reserved.

[그림 37] cars 데이터의 잔차도 — 적합값이 커질수록 잔차가 넓게 퍼진다(등분산성 의심).

7.2.2 빨간 추세선이 0 선을 따라가는지

`plot(fit)`은 네 개의 진단 플롯을 한꺼번에 보여준다 — Residuals vs Fitted, Normal Q-Q, Scale-Location, Residuals vs Leverage. 처음 둘만 자세히 보고, 나머지는 이상점 진단용 정도로 가볍게 본다.

진단 플롯 생성

```
import matplotlib.pyplot as plt
from scipy import stats
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
```

1) Residuals vs Fitted

```
residuals = model_sm.resid
fitted = model_sm.fittedvalues
axes[0, 0].scatter(fitted, residuals)
axes[0, 0].axhline(y=0, color='r', linestyle='--')
axes[0, 0].set_title('Residuals vs Fitted')
```

2) Q-Q Plot

```
stats.probplot(residuals, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Normal Q-Q')
```

```
# 3) Scale-Location
axes[1, 0].scatter(fitted, np.sqrt(np.abs(residuals)))
axes[1, 0].set_title('Scale-Location')

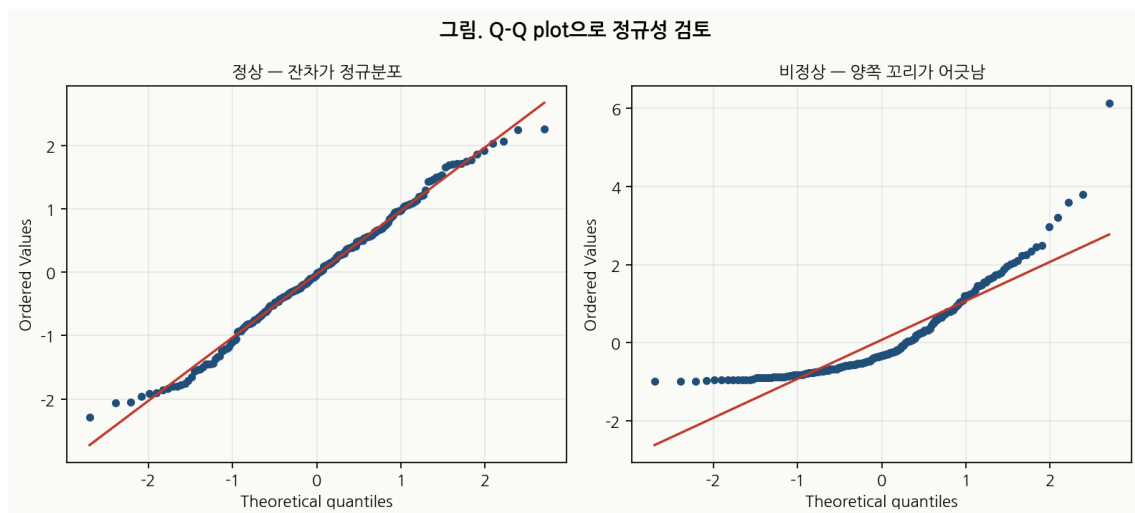
# 4) Residuals vs Leverage
from statsmodels.graphics.gofplots import OLSInfluence
influence = OLSInfluence(model_sm)
leverage = influence.hat_matrix_diag
axes[1, 1].scatter(leverage, residuals)
axes[1, 1].axhline(y=0, color='r', linestyle='--')
axes[1, 1].set_title('Residuals vs Leverage')

plt.tight_layout()
plt.show()
```

7.3 정규성 검정 – Q-Q plot

7.3.1 분위수 대조도 읽는 법

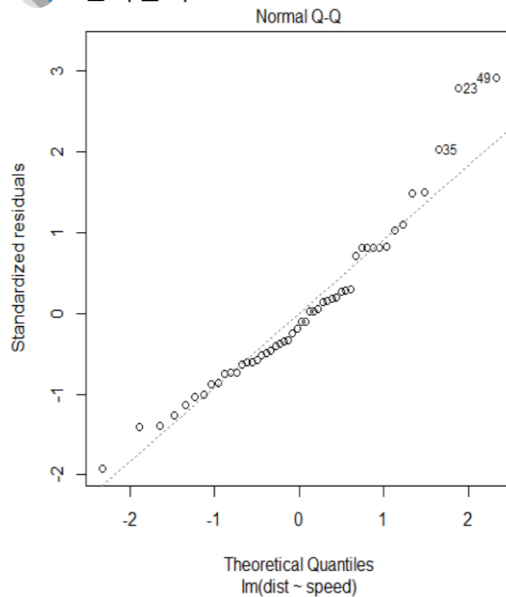
Q-Q plot(분위수 대조도, Quantile-Quantile plot)은 잔차의 분포가 정규분포와 얼마나 닮았는지 시각적으로 확인하는 도구다. 점들이 직선 위에 잘 놓이면 정규성을 만족하는 것이고, 양쪽 꼬리가 직선에서 크게 벗어나면 의심해 봐야 한다.



[그림 38] Q-Q plot 비교 — 왼쪽은 정상, 오른쪽은 양쪽 꼬리가 어긋난 비정상.



잔차분석



```
> plot(fit.cars, 2)
```

정규분포 분위수대조도

(Quantile-Quantile plot)를

확인해보면 양 끝쪽으로

직선에서 벗어난 점들이 관찰

되는데

대부분의 잔차는 직선 주위에

몰려있는 것을 확인할 수 있다.

© Copyrights 2018. WeDataLab All Rights Reserved.

[그림 39] cars 데이터 잔차의 Q-Q plot — 양 끝 점들이 직선에서 살짝 벗어나지만 대부분은 직선 위에 있다.

7.3.2 shapiro.test()의 p-value 해석

```
# 정규성 검정 (Shapiro-Wilk Test)
from scipy.stats import shapiro

stat, p_value = shapiro(residuals)
print(f'Shapiro-Wilk Test:')
print(f' 통계량: {stat:.4f}')
print(f' p-값: {p_value:.4f}')
if p_value > 0.05:
    print(' → 정규성 가정 만족 (p > 0.05)')
else:
    print(' → 정규성 가정 위반 (p < 0.05)')
```

Shapiro-Wilk 검정의 귀무가설은 '잔차가 정규분포다'이다. 따라서 p-value가 작으면 (보통 0.05 미만) 정규성 가정을 의심해야 한다. cars의 잔차는 $p \approx 0.022$ 로, 엄밀하게는 정규성을 약간 벗어난다.

7.4 등분산성 검정

잔차도에서 깔때기 모양이 보이면 등분산성을 의심한다. 시각적 판단을 보강하기 위해 정량적 검정도 함께 쓴다. Breusch-Pagan Test()가 대표적이다.

```
# 등분산성 검정 (Breusch-Pagan Test)
from statsmodels.stats.diagnostic import het_breuschpagan

bp_stat, bp_pval, _, _ = het_breuschpagan(residuals, X_const)
```

```

print(f'Breusch-Pagan Test:')
print(f' 통계량: {bp_stat:.4f}')
print(f' p-값: {bp_pval:.4f}')
if bp_pval > 0.05:
    print(' → 등분산성 가정 만족 ( $p > 0.05$ )')
else:
    print(' → 등분산성 가정 위반 ( $p < 0.05$ )')

```

Breusch-Pagan Test()의 귀무가설은 '등분산이다'. cars 데이터에서 p-value 가 0.073 으로 0.05 근처라 애매한 결과다. 잔차도의 깔때기 모양과 합쳐 보면 의심의 여지가 충분하다.

7.5 가정이 깨졌을 때 — 선형회귀에서 머신러닝으로

7.5.1 비선형 패턴은 SVM·트리·신경망의 영역

잔차분석 결과 선형성이 깨졌다면, 이는 데이터가 선형이 아니라 비선형 경향을 가지고 있다는 신호다. 서포트벡터머신(SVM), 의사결정나무, 신경망 등 머신러닝의 수많은 기법들은 바로 이런 복잡한 비선형 경향성을 가정하고 이를 학습하기 위해 개발된 것이다.

7.5.2 단, 변환만으로 살릴 수 있는 경우도 많다

그렇다고 곧장 머신러닝으로 점프해야 하는 것은 아니다. 단순한 데이터 변환(로그·제곱근·역수 등)으로 선형성과 등분산성을 동시에 살릴 수 있는 경우가 많다. 다음 8 장에서 cars 데이터에 제곱근 변환을 적용해 회귀를 다시 하는 사례를 본다.

7 장 정리

잔차도(Residuals vs Fitted): 패턴 없음 = OK, 깔때기 = 등분산성 깨짐.

Q-Q plot 과 shapiro.test()로 정규성 검토, . Breusch-Pagan Test()로 등분산성 검토.

p-value 가 작으면 해당 가정을 의심.

가정이 깨지면 변환(8 장) 또는 비선형 모델(머신러닝)으로 대응.

8 장. 회귀분석 다시 하기 — 데이터 변환

cars 데이터에서 잔차분석 결과 등분산성과 정규성에 약점이 있었다. 모형을 통째로 버리는 대신 종속변수에 간단한 변환을 적용해 가정을 살리는 길이 있다. 본 장에서는 제곱근 변환을 사용해 cars 데이터의 회귀를 다시 하고, 그 효과가 단지 통계적 트릭이 아니라 물리 법칙과도 맞아 떨어진다는 점을 확인한다.

8.1 왜 제곱근 변환인가

8.1.1 cars 데이터의 등분산성·정규성 위배 사례

잔차도에서 적합값이 커질수록 잔차가 넓게 퍼지는 깔때기 패턴이 보였다(그림 7-2). 이는 y 의 분산이 평균에 비례해 커지는 분포 — 예컨대 포아송 비슷한 분포 — 일 때 흔히 일어난다. 이런 경우 $\sqrt{y}$ 변환이 분산을 안정화시키는 고전적인 처방이다.

8.1.2 종속변수에 $\sqrt{\cdot}$ 를 씌우는 이유

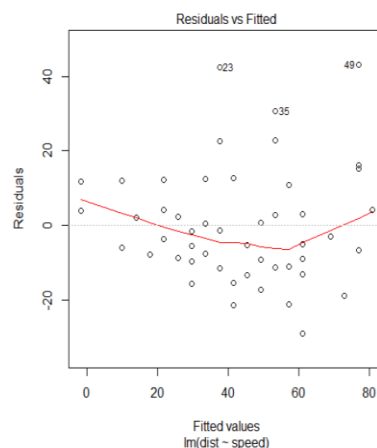
y 의 분산이 평균에 비례한다고 가정해 보자. 즉 $\text{Var}(y) = c \cdot E[y]$. 이때 분산안정화 변환(variance-stabilizing transformation)을 적용하면 $\text{Var}(\sqrt{y}) \approx c/4$ 로 평균에 무관한 상수가 된다. 즉 $\sqrt{y}$ 는 분산을 평탄하게 만든다. 통계학자 Box-Cox가 일반화시킨 변환 일가의 한 특수형이다.



데이터 변환

주어진 데이터는 $\hat{y}$ 의 값이 커질수록 잔차가 커지는 경향이 있었다.

또한 정규성 가정도 문제가 있었다.



[그림 40] 데이터 변환 개념 .

8.2 변환 후 다시 적합

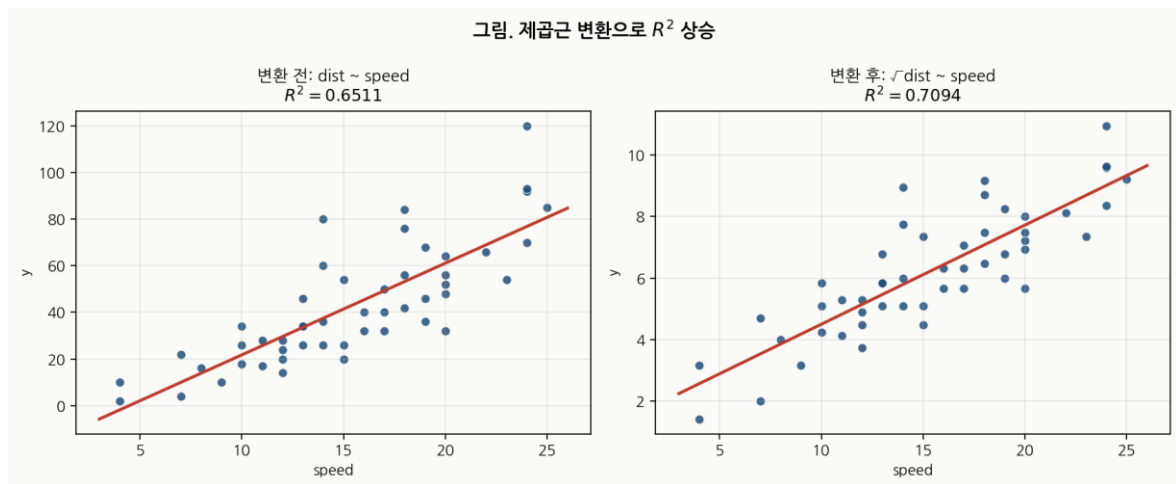
8.2.1 sqrt(dist) ~ speed 의 결과

```
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

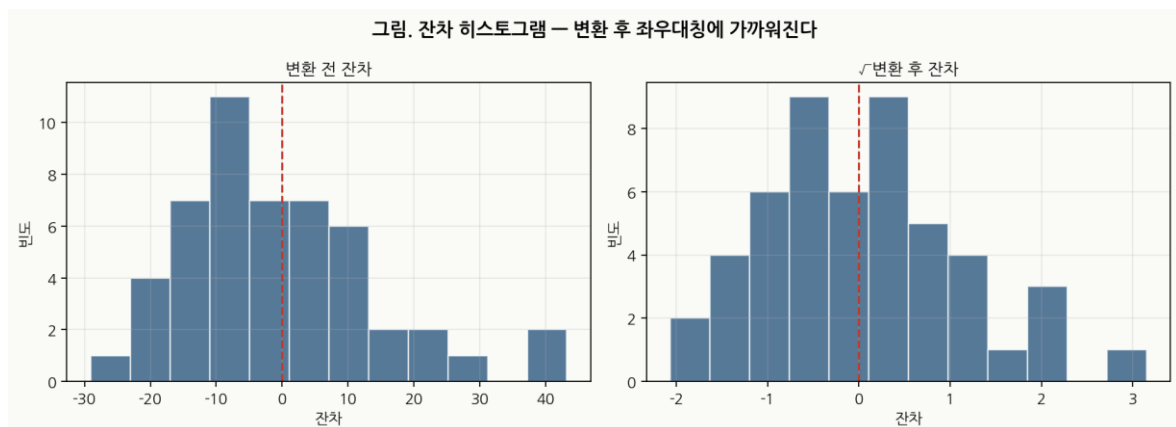
# 1. 데이터 준비
cars = sm.datasets.get_rdataset("cars", "datasets").data

# 2. 모델 정의 및 학습 (R: lm(sqrt(dist) ~ speed, data = cars))
# 포물러 안에 np.sqrt()를 직접 사용할 수 있습니다.
fit2 = smf.ols('np.sqrt(dist) ~ speed', data=cars).fit()

# 3. 상세 요약 보고서 출력 (R: summary(fit2))
print(fit2.summary())
```



[그림 41] 변환 전(왼쪽)과 $\sqrt{\text{변환}}$ 후(오른쪽). 점들이 직선에 더 잘 붙고 R^2 가 상승한다.



[그림 42] 잔차 히스토그램 — $\sqrt{\text{변환}}$ 후 좌우대칭에 더 가까워진다.

8.2.2 결정계수가 0.6511 → 0.7094 로 상승



제공된 변환을 시행한 자료에 선형회귀모형 적합

```
> summary(fit.cars2)
```

```
Call:
lm(formula = sqrt.dist ~ speed, data = data2)
```

```
Residuals:
    Min       1Q   Median       3Q      Max
-2.0684 -0.6983 -0.1799  0.5909  3.1534
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.27705    0.48444   2.636  0.0113 *
speed        0.32241    0.02978  10.825 1.77e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

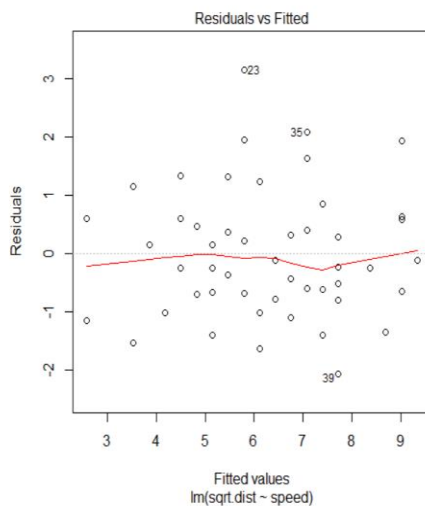
```
Residual standard error: 1.102 on 48 degrees of freedom
Multiple R-squared:  0.7094,    Adjusted R-squared:  0.7034
F-statistic: 117.2 on 1 and 48 DF, p-value: 1.773e-14
```

0.6511 -> 0.7094로
결정계수 증가

[그림 43] 변환으로 R^2 이 0.6511에서 0.7094로 상승.



제공된 변환을 시행한 자료에 선형회귀모형 적합



```
> bptest(fit.cars2)
```

studentized Breusch-Pagan test

```
data: fit.cars2
BP = 0.011192, df = 1, p-value = 0.9157
```

```
> ncvTest(fit.cars2)
```

```
Non-constant Variance Score Test
Variance formula: ~ fitted.values
Chisquare = 0.01205185 Df = 1
```

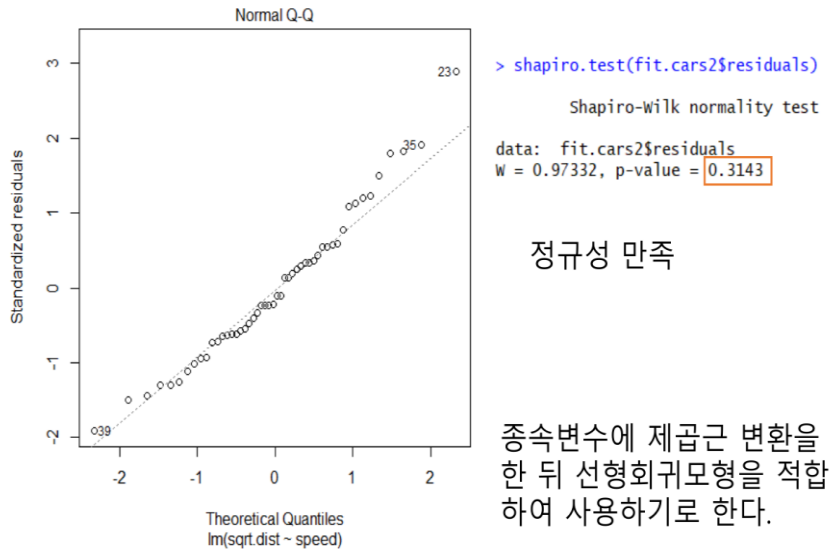
p = 0.9125831

등분산성 만족

[그림 44] 변환 후 잔차도 — 등분산성이 만족스럽게 회복되었다.



제곱근 변환을 시행한 자료에 선형회귀모형 적합



[그림 45] 변환 후 Q-Q plot — 점들이 직선에 거의 붙어 있어 정규성 만족.

8.3 물리적 의미 — 속도의 제곱

8.3.1 등가속도 운동과 제동거리 공식

물리 법칙으로 보면 제동거리는 운동 에너지를 마찰력이 소비할 때까지의 거리이므로, 속도의 제곱에 비례한다($v^2 = 2as$ 의 a, s 관계).

[물리 공식] 등가속도 제동거리

$$v^2 = v_0^2 - 2 \cdot a \cdot s \quad (v=0 \text{ 에서 정지})$$

$$\rightarrow s = v_0^2 / (2 \cdot a)$$

즉 제동거리 s 는 초기 속도 v_0 의 '제곱'에 비례.

$$\text{dist} \propto \text{speed}^2 \Leftrightarrow \sqrt{\text{dist}} \propto \text{speed}$$

8.3.2 통계적 변환이 물리 법칙과 맞아떨어진 사례

통계적으로는 등분산성을 회복하기 위해 $\sqrt{y}$ 변환을 도입했는데, 그 결과 모형 $\sqrt{\text{dist}} \approx a + b \cdot \text{speed}$ 가 되었다. 이를 양변 제곱하면 $\text{dist} \approx (a + b \cdot \text{speed})^2$ 가 되는데, 큰 speed에서 이는 사실상 $\text{dist} \propto \text{speed}^2$ 와 같다. 즉 통계의 처방과 물리의 법칙이 일치한 셈이다. 이런 사례는 회귀 변환이 단순한 트릭이 아니라 데이터의 본질을 드러내는 도구임을 보여 준다.



(참고) 속력과 제동 거리의 관계

제동거리는 브레이크를 밟은 후 정지할 때까지 이동한 거리이다. 이 때는 속도가 점점 줄어들기 때문에 등가속도 운동이라고 한다. 이 때 움직인 거리는 다음 공식으로 구하게 된다.

$$(\text{움직인 거리}) = \frac{(\text{나중 속도})^2 - (\text{처음 속도})^2}{2 \times (\text{가속도})}$$

이론상 제동 거리는 속도가 아닌 속도의 제곱과 관계가 있으므로, 제곱근 변환을 해준 뒤 선형모형을 적합시킨 것은 적절했다고 볼 수 있다.

[그림 46] 속력과 제동거리의 물리적 관계 .

8.4 다른 변환 방식들

8.4.1 로그 변환·Box-Cox·역수 변환

y의 분포에 따라 자주 쓰는 변환들이 있다. 대략적으로 분류하면 다음과 같다.

[변환 종류] 자주 쓰는 종속변수 변환

- 로그 변환 $\ln(y)$ — y가 곱연산적 변동성을 갖거나(가격, 인구) 우측 꼬리가 긴 경우
- 제곱근 변환 $\sqrt{y}$ — y의 분산이 평균에 비례할 때(포아송 비슷한 카운트 데이터)
- 역수 변환 $1/y$ — y가 매우 빠르게 증가/감소하는 비율 자료
- Box-Cox 변환 — 위 세 변환을 한 모수 λ 로 통합한 일반화.

8.4.2 어떤 변환을 언제 쓰는가

2. 최적의 람다(lambda) 값 추정

`scipy.stats.boxcox_normmax`는 종속변수(y)를 입력받아 최적의 λ 를 계산합니다.

```
lambda_hat = stats.boxcox_normmax(cars['dist'], method='mle')
print(f"lambda_hat: {lambda_hat:.2f}")
```

3. 결론 출력

```
print(f"→  $\lambda \approx \{\text{round}(\text{lambda\_hat} * 2) / 2\}$ 이므로 제곱근 변환(sqrt)이 자연스러운 선택")
```

Box-Cox는 데이터에 가장 잘 맞는 변환 모수 λ 를 자동으로 추정해 준다. $\lambda=1$ 이면 변환 없음, $\lambda=0$ 이면 로그 변환, $\lambda=0.5$ 이면 제곱근 변환, $\lambda=-1$ 이면 역수 변환에 해당한다. cars 데이터의 $\hat{\lambda}$ 는 0.42 부근으로, 0.5(제곱근)에 가까운 값이 나오므로 우리가 $\sqrt{}$ 를 선택한 것은 통계적으로도 정당화된다.

팁 · 변환 후 해석할 때 주의할 것

변환된 y의 회귀계수를 그대로 '원래 단위'로 해석하면 안 된다.

$\sqrt{}$ 변환을 했다면 예측값을 다시 제곱해서 원래 단위로 되돌려야 한다.

그 과정에서 신뢰구간도 비대칭이 되므로, `predict()` 결과의 `lwr·upr` 도 함께 제공해 줘야 한다.

8 장 정리

잔차에 깔때기 패턴이 보이면 $\sqrt{y}$ 변환을 1 순위로 시도.

cars 데이터: $\sqrt{\text{dist}} \sim \text{speed}$ 로 R^2 가 0.6511 $\rightarrow$ 0.7094 상승, 등분산성·정규성도 회복.

이는 물리 법칙($\text{dist} \propto \text{speed}^2$)과도 일치 — 통계 처방이 도메인 진실과 맞아떨어진 사례.

Box-Cox 로 λ 를 자동 추정해 변환의 적절성을 정량 확인할 수 있다.

9 장. 정리 및 다음 단계

9.1 단순선형회귀 한 줄 요약

단순선형회귀를 단 한 줄로

$y = \beta_0 + \beta_1 x + \epsilon$. OLS 로 β_0, β_1 을 잔차제곱합 최소화로 추정. 가정 검토는 잔차분석으로.

이 강의 자료는 단순선형회귀라는 가장 단순한 모델을 가능한 한 깊게 들여다보는 것을 목표로 했다. 회귀의 어원부터 시작해 OLS 의 수식 유도, R 실습, 잔차분석, 변환까지 — 이 과정이 모두 단 하나의 직선식 $y = \beta_0 + \beta_1 x + \epsilon$ 안에 들어 있다는 사실이 신기할 정도다.

수고하셨습니다

단순선형회귀는 모든 통계 모델의 출발점이다.

이 모델을 손에 익히면, 그 위에 다중회귀·로지스틱·일반화선형모형·트리 모델까지 자연스럽게 쌓아 올릴 수 있다.

정리 및 요약

여기까지가 단순선형회귀의 전 과정이다. 모형을 만들고 $\rightarrow$ 회귀계수를 추정하고 $\rightarrow$ 적합도를 평가하고 $\rightarrow$ 가정을 검토하고 $\rightarrow$ 가정이 깨지면 변환으로 개선한다. 이 한 사이클이 사실상 모든 회귀분석의 표준 흐름이다.

핵심 개념 정리

- 회귀의 어원 — 골턴의 "평균으로의 회귀". 지금은 일반적인 선형 관계 추정 기법을 가리킨다.

- 변수 이름 사전 — x 는 독립/설명/예측/입력/특징, y 는 종속/반응/결과/출력/타겟. 분야마다 다름.
- 상관 vs 회귀 — 상관은 같이 움직이는 정도(대칭), 회귀는 $x \rightarrow y$ 방향성(비대칭). 둘 다 인과는 모름.
- 모형식 — $y = \beta_0 + \beta_1 x + \epsilon$. ϵ 은 이론적 오차, $e = y - \hat{y}$ 는 데이터에서 계산되는 잔차.
- OLS — y 축 거리 제곱합 최소화. 닫힌 형태 해 존재. 이상치에 약함.
- β_1 해석 — x 가 1 단위 증가할 때 y 의 평균 변화량. 단위 중요.
- p-value — "계수가 0 이라는 가설" 이 우연일 확률. 0.05 미만이면 유의. 효과 크기와는 다르다.
- $R^2 = SSR/SST$ — y 변동 중 회귀선이 설명한 비율. 0~1. 다중회귀에서는 Adjusted R^2 도 함께.
- 가정 6 가지 — 선형성·정규성·등분산성·독립성·Zero-mean·다중공선성. 단순회귀는 4 가지가 핵심.
- 잔차 4 플롯 — Residuals vs Fitted / Normal Q-Q / Scale-Location / Residuals vs Leverage.
- 정규성 검정 — Shapiro-Wilk. 등분산성 검정 — Breusch-Pagan.
- 가정 위반 처방 — 변환($\sqrt{\cdot}$, log, Box-Cox), 가중회귀(WLS), robust 표준오차, 비선형 회귀.
- Python 두 도구 — sklearn LinearRegression (빠른 학습·예측) + statsmodels OLS (통계 요약).

과제

아래 5 문제는 본 강의 전체를 종합적으로 점검하는 문제다. 자기 손으로 풀고, 코드 문제는 직접 실행해 보자.

1. "회귀(regression)" 라는 이름의 어원을 골턴의 발견(부모-자녀 키 연구) 으로부터 설명하고, 현재의 회귀분석이 그 어원과 어떻게 다른 의미로 쓰이는지 적어라.
2. 회귀계수표에서 $\beta_1 = 3.9324$, β_1 의 p-값 = $1.49e-12$ 가 나왔다. β_1 의 의미를 단위와 함께 한 문장으로 설명하고, 이 결과의 통계적 유의성을 0.05 기준으로 판단하라.
3. cars 데이터에 단순선형회귀를 적용하니 $R^2 = 0.6511$ 이 나왔다. 이 값의 의미를 $SST/SSR/SSE$ 의 관계식으로 설명하고, "양호한 적합" 인지 판단하라.

4. (코드) sklearn 의 load_diabetes() 데이터에서 "bmi" 변수 하나만 골라 단순선형회귀로 disease progression 을 예측하라. 회귀계수 β_1 , R^2 를 출력하고 산점도+회귀선을 그려라. statsmodels 로 p-value 도 함께 확인하라.
5. (응용) cars 데이터에 단순회귀를 적용하면 Breusch-Pagan p-값이 0.05 미만으로 등분산성 가정이 위반된다. $\sqrt{\text{dist}}$ 로 변환 후 다시 회귀했더니 등분산성이 회복되었다. 왜 제곱근 변환이 이 데이터에서 효과적인지 "운동에너지" 키워드로 설명하라.

과제 해답

1 번 해답

골턴은 부모의 키와 자녀의 키를 측정해 보고, "키 큰 부모의 자녀는 부모보다 약간 작아지고, 키 작은 부모의 자녀는 부모보다 약간 커지는" 경향을 발견했다. 자녀의 키가 인류 평균 쪽으로 "회귀(regression toward the mean)" 하는 것처럼 보였기에 이 현상에 "regression" 이라는 이름을 붙였다. 현재의 회귀분석은 이 어원과 달리 "평균으로 돌아간다" 는 뜻이 아니라, "한 변수에서 다른 변수로 이어지는 선형 관계를 추정하는 일반적 분석 기법" 을 가리킨다. 이름만 남고 뜻은 일반화되었다.

2 번 해답

$\beta_1 = 3.9324$ 의 의미 — speed(자동차 속도) 가 1 mph 증가할 때 dist(제동 거리) 가 평균적으로 3.9324 ft 증가한다. 즉 단위는 "ft per mph". 통계적 유의성: p-값 1.49e-12 는 0.05 보다 압도적으로 작아 β_1 이 0 이라는 귀무가설을 강하게 기각한다. 즉 speed 는 dist 와 통계적으로 매우 유의한 양의 선형 관계를 갖는다.

3 번 해답

$R^2 = SSR/SST = 1 - SSE/SST$. $R^2 = 0.6511$ 은 cars 데이터에서 dist 의 전체 변동(SST) 중 약 65.11% 를 speed 라는 단일 변수의 회귀선이 설명하고, 나머지 34.89% 는 회귀선이 설명하지 못한다는 뜻이다. 일반 기준으로 0.5~0.7 은 "보통의 설명력" 이며, 사회·실무 데이터에서는 양호하다고 본다. 다만 잔차분석을 통과해야 비로소 "신뢰 가능한" 적합으로 인정된다.

4 번 해답

코드: load_diabetes() 로 데이터를 불러와 bmi 변수만 (-1,1) 모양으로 추려

LinearRegression 에 fit. $\beta_1 \approx 949$, $R^2 \approx 0.34$, p-값 $\approx 3e-42$ 로 매우 유의. 산점도에 회귀선을 빨간선으로 얹으면 BMI 가 클수록 진행도가 빨라지는 명백한 양의 관계가 보인다.


```

import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression

# 1. 데이터 로드 및 전처리
db = load_diabetes()
bmi_index = db.feature_names.index("bmi")

X = db.data[:, bmi_index].reshape(-1, 1)
y = db.target

# 2. Scikit-learn 을 이용한 선형 회귀 분석
model = LinearRegression().fit(X, y)
print(f" $\hat{\beta}_1 = \{{model.coef_[0]:.4f}\}$ ,  $R^2 = \{{model.score(X, y):.4f}\}$ ")
print("-" * 60)

# 3. Statsmodels 를 이용한 상세 통계 분석
X_with_constant = sm.add_constant(X)
sm_model = sm.OLS(y, X_with_constant).fit()
print(sm_model.summary())

# 4. 시각화 (산점도 및 회귀선)
plt.figure(figsize=(8, 6))
plt.scatter(X, y, alpha=0.5, label="Data")
plt.plot(X, model.predict(X), "r-", linewidth=2, label="Regression Line")
plt.title("Diabetes Dataset: BMI vs Target", fontsize=14)
plt.xlabel("BMI (Standardized)", fontsize=12)
plt.ylabel("Target", fontsize=12)
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

```

5 번 해답

제동거리(dist) 와 속도(speed) 의 관계는 직선이 아니라 " $\text{dist} \propto \text{speed}^2$ " 다. 이유는 물리적으로 명확하다 — 자동차의 운동에너지는 $\frac{1}{2}mv^2$ 이고, 정지하기 위해 마찰력이 해야 할 일은 운동에너지와 같아야 한다. 마찰력이 일정하다면 일 = 힘 $\times$ 거리 이므로 제동거리는 v^2 에 비례한다. 따라서 $\sqrt{\text{dist}}$ 를 종속변수로 두면 speed 와 직선 관계가 된다. 이 변환이 등분산성을 회복시키는 이유도 같다 — 원래 데이터에서 speed 가 커질수록 분산이 함께 커지던 것이, $\sqrt{\text{dist}}$ 로 바꾸면 분산이 평탄해진다.